NAME

ovn-northd − Open Virtual Network central control daemon

SYNOPSIS

ovn−northd [*options*]

DESCRIPTION

ovn−northd is a centralized daemon responsible for translating the high-level OVN configuration into logical configuration consumable by daemons such as **ovn−controller**. It translates the logical network configuration in terms of conventional network concepts, taken from the OVN Northbound Database (see **ovn−nb**(5)), into logical datapath flows in the OVN Southbound Database (see **ovn−sb**(5)) below it.

OPTIONS

−−ovnnb−db=*database*

The OVSDB database containing the OVN Northbound Database. If the **OVN_NB_DB** environment variable is set, its value is used as the default. Otherwise, the default is **unix:/ovnnb_db.sock**.

−−ovnsb−db=*database*

The OVSDB database containing the OVN Southbound Database. If the **OVN_SB_DB** environment variable is set, its value is used as the default. Otherwise, the default is **unix:/ovnsb_db.sock**.

−−dry−run

Causes **ovn−northd** to start paused. In the paused state, **ovn−northd** does not apply any changes to the databases, although it continues to monitor them. For more information, see the **pause** command, under **Runtime Management Commands** below.

n−threads N

In certain situations, it may be desirable to enable parallelization on a system to decrease latency (at the potential cost of increasing CPU usage).

This option will cause ovn-northd to use N threads when building logical flows, when N is within [2−256]. If N is 1, parallelization is disabled (default behavior). If N is less than 1, then N is set to 1, parallelization is disabled and a warning is logged. If N is more than 256, then N is set to 256, parallelization is enabled (with 256 threads) and a warning is logged.

database in the above options must be an OVSDB active or passive connection method, as described in **ovsdb**(7).

Daemon Options

−−pidfile[**=***pidfile*]

Causes a file (by default, *program***.pid**) to be created indicating the PID of the running process. If the *pidfile* argument is not specified, or if it does not begin with **/**, then it is created in .

If **−−pidfile** is not specified, no pidfile is created.

−−overwrite−pidfile

By default, when **−−pidfile** is specified and the specified pidfile already exists and is locked by a running process, the daemon refuses to start. Specify **−−overwrite−pidfile** to cause it to instead overwrite the pidfile.

When **−−pidfile** is not specified, this option has no effect.

−−detach

Runs this program as a background process. The process forks, and in the child it starts a new session, closes the standard file descriptors (which has the side effect of disabling logging to the console), and changes its current directory to the root (unless **−−no−chdir** is specified). After the child completes its initialization, the parent exits.

−−monitor

Creates an additional process to monitor this program. If it dies due to a signal that indicates a programming error (**SIGABRT**, **SIGALRM**, **SIGBUS**, **SIGFPE**, **SIGILL**, **SIGPIPE**, **SIGSEGV**,

SIGXCPU, or **SIGXFSZ**) then the monitor process starts a new copy of it. If the daemon dies or exits for another reason, the monitor process exits.

This option is normally used with **−−detach**, but it also functions without it.

−−no−chdir

By default, when **−−detach** is specified, the daemon changes its current working directory to the root directory after it detaches. Otherwise, invoking the daemon from a carelessly chosen directory would prevent the administrator from unmounting the file system that holds that directory.

Specifying **−−no−chdir** suppresses this behavior, preventing the daemon from changing its current working directory. This may be useful for collecting core files, since it is common behavior to write core dumps into the current working directory and the root directory is not a good directory to use.

This option has no effect when **−−detach** is not specified.

−−no−self−confinement

By default this daemon will try to self-confine itself to work with files under well-known directories determined at build time. It is better to stick with this default behavior and not to use this flag unless some other Access Control is used to confine daemon. Note that in contrast to other access control implementations that are typically enforced from kernel-space (e.g. DAC or MAC), self-confinement is imposed from the user-space daemon itself and hence should not be considered as a full confinement strategy, but instead should be viewed as an additional layer of security.

−−user=*user***:***group*

Causes this program to run as a different user specified in *user***:***group*, thus dropping most of the root privileges. Short forms *user* and **:***group* are also allowed, with current user or group assumed, respectively. Only daemons started by the root user accepts this argument.

On Linux, daemons will be granted **CAP_IPC_LOCK** and **CAP_NET_BIND_SERVICES** before dropping root privileges. Daemons that interact with a datapath, such as **ovs−vswitchd**, will be granted three additional capabilities, namely **CAP_NET_ADMIN**, **CAP_NET_BROADCAST** and **CAP_NET_RAW**. The capability change will apply even if the new user is root.

On Windows, this option is not currently supported. For security reasons, specifying this option will cause the daemon process not to start.

Logging Options

−v[*spec*]

−−verbose=[*spec*]

Sets logging levels. Without any *spec*, sets the log level for every module and destination to **dbg**. Otherwise, *spec* is a list of words separated by spaces or commas or colons, up to one from each category below:

- A valid module name, as displayed by the **vlog/list** command on **ovs−appctl**(8), limits the log level change to the specified module.
- **syslog**, **console**, or **file**, to limit the log level change to only to the system log, to the console, or to a file, respectively. (If **−−detach** is specified, the daemon closes its standard file descriptors, so logging to the console will have no effect.)

  On Windows platform, **syslog** is accepted as a word and is only useful along with the **−−syslog−target** option (the word has no effect otherwise).
- **off**, **emer**, **err**, **warn**, **info**, or **dbg**, to control the log level. Messages of the given severity or higher will be logged, and messages of lower severity will be filtered out. **off** filters out all messages. See **ovs−appctl**(8) for a definition of each log level.

Case is not significant within *spec*.

Regardless of the log levels set for **file**, logging to a file will not take place unless **−−log−file** is also specified (see below).

For compatibility with older versions of OVS, **any** is accepted as a word but has no effect.

−v
−−verbose
Sets the maximum logging verbosity level, equivalent to **−−verbose=dbg**.

−vPATTERN:*destination***:***pattern*
−−verbose=PATTERN:*destination***:***pattern*
Sets the log pattern for *destination* to *pattern*. Refer to **ovs−appctl**(8) for a description of the valid syntax for *pattern*.

−vFACILITY:*facility*
−−verbose=FACILITY:*facility*
Sets the RFC5424 facility of the log message. *facility* can be one of **kern**, **user**, **mail**, **daemon**, **auth**, **syslog**, **lpr**, **news**, **uucp**, **clock**, **ftp**, **ntp**, **audit**, **alert**, **clock2**, **local0**, **local1**, **local2**, **local3**, **local4**, **local5**, **local6** or **local7**. If this option is not specified, **daemon** is used as the default for the local system syslog and **local0** is used while sending a message to the target provided via the **−−syslog−target** option.

−−log−file[**=***file*]
Enables logging to a file. If *file* is specified, then it is used as the exact name for the log file. The default log file name used if *file* is omitted is **/usr/local/var/log/ovn/***program***.log**.

−−syslog−target=*host***:***port*
Send syslog messages to UDP *port* on *host*, in addition to the system syslog. The *host* must be a numerical IP address, not a hostname.

−−syslog−method=*method*
Specify *method* as how syslog messages should be sent to syslog daemon. The following forms are supported:

- **libc**, to use the libc **syslog()** function. Downside of using this options is that libc adds fixed prefix to every message before it is actually sent to the syslog daemon over **/dev/log** UNIX domain socket.
- **unix:***file*, to use a UNIX domain socket directly. It is possible to specify arbitrary message format with this option. However, **rsyslogd 8.9** and older versions use hard coded parser function anyway that limits UNIX domain socket use. If you want to use arbitrary message format with older **rsyslogd** versions, then use UDP socket to localhost IP address instead.
- **udp:***ip***:***port*, to use a UDP socket. With this method it is possible to use arbitrary message format also with older **rsyslogd**. When sending syslog messages over UDP socket extra precaution needs to be taken into account, for example, syslog daemon needs to be configured to listen on the specified UDP port, accidental iptables rules could be interfering with local syslog traffic and there are some security considerations that apply to UDP sockets, but do not apply to UNIX domain sockets.
- **null**, to discard all messages logged to syslog.

The default is taken from the **OVS_SYSLOG_METHOD** environment variable; if it is unset, the default is **libc**.

PKI Options

PKI configuration is required in order to use SSL/TLS for the connections to the Northbound and Southbound databases.

−p *privkey.pem*
−−private−key=*privkey.pem*
Specifies a PEM file containing the private key used as identity for outgoing SSL/TLS connections.

−c *cert.pem*
−−certificate=*cert.pem*
Specifies a PEM file containing a certificate that certifies the private key specified on **−p** or **−−private−key** to be trustworthy. The certificate must be signed by the certificate authority (CA) that the peer in SSL/TLS connections will use to verify it.

−C *cacert.pem*
−−ca−cert=*cacert.pem*
Specifies a PEM file containing the CA certificate for verifying certificates presented to this program by SSL/TLS peers. (This may be the same certificate that SSL/TLS peers use to verify the certificate specified on **−c** or **−−certificate**, or it may be a different one, depending on the PKI design in use.)

−C none
−−ca−cert=none
Disables verification of certificates presented by SSL/TLS peers. This introduces a security risk, because it means that certificates cannot be verified to be those of known trusted hosts.

−−ssl−server−name=*servername*
Specifies the server name to use for TLS Server Name Indication (SNI). By default, the hostname from the connection string is used for SNI. This option allows overriding the SNI hostname, which is useful when connecting through proxies or service meshes where the connection endpoint differs from the intended server name.

Other Options

−−unixctl=*socket*
Sets the name of the control socket on which *program* listens for runtime management commands (see *RUNTIME MANAGEMENT COMMANDS,* below). If *socket* does not begin with **/**, it is interpreted as relative to . If **−−unixctl** is not used at all, the default socket is **/***program***.***pid***.ctl**, where *pid* is *program*'s process ID.

On Windows a local named pipe is used to listen for runtime management commands. A file is created in the absolute path as pointed by *socket* or if **−−unixctl** is not used at all, a file is created as *program* in the configured *OVS_RUNDIR* directory. The file exists just to mimic the behavior of a Unix domain socket.

Specifying **none** for *socket* disables the control socket feature.

−h
−−help Prints a brief help message to the console.

−V
−−version
Prints version information to the console.

RUNTIME MANAGEMENT COMMANDS

ovn−appctl can send commands to a running **ovn−northd** process. The currently supported commands are described below.

exit Causes **ovn−northd** to gracefully terminate.

pause Pauses **ovn−northd**. When it is paused, **ovn−northd** receives changes from the Northbound and Southbound database changes as usual, but it does not send any updates. A paused **ovn−northd** also drops database locks, which allows any other non-paused instance of **ovn−northd** to take over.

resume Resumes the ovn-northd operation to process Northbound and Southbound database contents and generate logical flows. This will also instruct ovn-northd to aspire for the lock on SB DB.

is−paused
Returns "true" if ovn-northd is currently paused, "false" otherwise.

status Prints this server's status. Status will be "active" if ovn-northd has acquired OVSDB lock on SB DB, "standby" if it has not or "paused" if this instance is paused.

sb−cluster−state−reset
Reset southbound database cluster status when databases are destroyed and rebuilt.

If all databases in a clustered southbound database are removed from disk, then the stored index of all databases will be reset to zero. This will cause ovn-northd to be unable to read or write to the southbound database, because it will always detect the data as stale. In such a case, run this command so that ovn-northd will reset its local index so that it can interact with the southbound database again.

nb−cluster−state−reset
Reset northbound database cluster status when databases are destroyed and rebuilt.

This performs the same task as **sb−cluster−state−reset** except for the northbound database client.

set−n−threads N
Set the number of threads used for building logical flows. When N is within [2−256], parallelization is enabled. When N is 1 parallelization is disabled. When N is less than 1 or more than 256, an error is returned. If ovn-northd fails to start parallelization (e.g. fails to setup semaphores, parallelization is disabled and an error is returned.

get−n−threads
Return the number of threads used for building logical flows.

inc−engine/show−stats
Display **ovn−northd** engine counters. For each engine node the following counters have been added:

- **recompute**
- **compute**
- **abort**

inc−engine/show−stats *engine_node_name counter_name*
Display the **ovn−northd** engine counter(s) for the specified *engine_node_name*. *counter_name* is optional and can be one of **recompute**, **compute** or **abort**.

inc−engine/clear−stats
Reset **ovn−northd** engine counters.

inc−engine/recompute
Triggers a full recompute of the incremental processing engine on the next iteration.

inc−engine/compute−log−timeout *msecs*
Sets the timeout in milliseconds for logging engine compute events.

inc−engine/list−stopwatches [*node*]
Lists the engine nodes and their change handler names. If *node* is specified, only the handlers for that engine node are listed.

inc−engine/enable−stopwatch [*node*]
Enables the per-node and per-handler stopwatches in the incremental processing engine. If *node* is specified, only that node's stopwatches are enabled; otherwise all nodes are affected. While enabled, every engine node recompute and change handler invocation is timed and the results can be viewed with the **stopwatch/show** command. Stopwatches are disabled by default because they add measurable CPU overhead.

inc−engine/disable−stopwatch [*node*]
Disables the per-node and per-handler stopwatches in the incremental processing engine. If *node* is specified, only that node's stopwatches are disabled; otherwise all nodes are affected. This is the default state.

ACTIVE-STANDBY FOR HIGH AVAILABILITY

You may run **ovn−northd** more than once in an OVN deployment. When connected to a standalone or clustered DB setup, OVN will automatically ensure that only one of them is active at a time. If multiple instances of **ovn−northd** are running and the active **ovn−northd** fails, one of the hot standby instances of **ovn−northd** will automatically take over.

Active−Standby with multiple OVN DB servers

You may run multiple OVN DB servers in an OVN deployment with:

- OVN DB servers deployed in active/passive mode with one active and multiple passive ovsdb-servers.
- **ovn−northd** also deployed on all these nodes, using unix ctl sockets to connect to the local OVN DB servers.

In such deployments, the ovn-northds on the passive nodes will process the DB changes and compute logical flows to be thrown out later, because write transactions are not allowed by the passive ovsdb-servers. It results in unnecessary CPU usage.

With the help of runtime management command **pause**, you can pause **ovn−northd** on these nodes. When a passive node becomes master, you can use the runtime management command **resume** to resume the **ovn−northd** to process the DB changes.

LOGICAL FLOW TABLE STRUCTURE

One of the main purposes of **ovn−northd** is to populate the **Logical_Flow** table in the **OVN_Southbound** database. This section describes how **ovn−northd** does this for switch and router logical datapaths.

Logical Switch Datapaths

Ingress Table 0: Admission Control and Ingress Port Security check

Ingress table 0 contains these logical flows:

- Priority 100 flows to drop packets with VLAN tags or multicast Ethernet source addresses.
- For each disabled logical port, a priority 100 flow is added which matches on all packets and applies the action **REGBIT_PORT_SEC_DROP" = 1; next;"** so that the packets are dropped in the next stage.
- For each logical port that's defined as a target of routing protocol redirecting (via **routing−protocol−redirect** option set on Logical Router Port), a filter is set in place that disallows following traffic exiting this port:
  - ARP replies
  - IPv6 Neighbor Discovery - Router Advertisements
  - IPv6 Neighbor Discovery - Neighbor Advertisements

  Since this port shares IP and MAC addresses with the Logical Router Port, we wan't to prevent duplicate replies and advertisements. This is achieved by a rule with priority 80 that sets **REGBIT_PORT_SEC_DROP" = 1; next;"**.
- For each (enabled) vtep logical port, a priority 70 flow is added which matches on all packets and applies the action **next(pipeline=ingress, table=S_SWITCH_IN_L3_LKUP) = 1;** to skip most stages of ingress pipeline and go directly to ingress L2 lookup table to determine the output port. Packets from VTEP (RAMP) switch should not be subjected to any ACL checks. Egress pipeline will do the ACL checks.

- For each enabled logical port configured with qdisc queue id in the **options:qdisc_queue_id** column of **Logical_Switch_Port**, a priority 70 flow is added which matches on all packets and applies the action **set_queue(id); REGBIT_PORT_SEC_DROP" = check_in_port_sec(); next;"**.
- A priority 1 flow is added which matches on all packets for all the logical ports and applies the action **REGBIT_PORT_SEC_DROP" = check_in_port_sec(); next;** to evaluate the port security. The action **check_in_port_sec** applies the port security rules defined in the **port_security** column of **Logical_Switch_Port** table.

Ingress Table 1: Ingress Port Security - Apply

For each logical switch port *P* of type router connected to a gw router a priority−120 flow that matches 'recirculated' icmp{4,6} error 'packet too big' and **eth.src == *D* && outport == *P* && flags.tunnel_rx == 1** where *D* is the peer logical router port *RP* mac address, swaps inport and outport and applies the action **next**.

For each logical switch port *P* of type router connected to a distributed router a priority−120 flow that matches 'recirculated' icmp{4,6} error 'packet too big' and **eth.dst == *D* && flags.tunnel_rx == 1** where *D* is the peer logical router port *RP* mac address, swaps inport and outport and applies the action **next(pipeline=S_SWITCH_IN_L2_LKUP)**.

For each logical switch port *P* a priority−110 flow that matches 'recirculated' icmp{4,6} error 'packet too big' and **eth.src == *D* && outport == *P* && !is_chassis_resident("*P*") && flags.tunnel_rx == 1** where *D* is the logical switch port mac address, swaps inport and outport and applies the action **next**.

This table adds a priority−105 flow that matches 'recirculated' icmp{4,6} error 'packet too big' to drop the packet.

This table drops the packets if the port security check failed in the previous stage i.e the register bit **REGBIT_PORT_SEC_DROP** is set to 1.

Ingress table 1 contains these logical flows:

- A priority−50 fallback flow that drops the packet if the register bit **REGBIT_PORT_SEC_DROP** is set to 1.
- One priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 2: Mirror

Overlay remote mirror table contains the following logical flows:

- For each logical switch port with an attached mirror, a logical flow with a priority of 100 is added. This flow matches all incoming packets to the attached port, clones them, and forwards the cloned packets to the mirror target port.
- A priority 0 flow is added which matches on all packets and applies the action **next;**.
- A logical flow added for each Mirror Rule in Mirror table attached to logical switch ports, matches all incoming packets that match rules and clones the packet and sends cloned packet to mirror target port.

Ingress Table 3: Lookup MAC address learning table

This table looks up the MAC learning table of the logical switch datapath to check if the **port−mac** pair is present or not. MAC is learnt for logical switch with 'unknown' address set as well as for localnet ports with option localnet_learn_fdb. A localnet port entry does not overwrite a VIF port entry. Logical switch ports with type **switch** have implicit 'unknown' addresses and so they are also eligible for MAC learning.

- For each such VIF logical port *p* with 'unknown' address set following flow is added.
  - Priority 120 flows for ARP packets with **arp.sha == 00:00:00:00:00:00**, ND packets with **nd.tll == 00:00:00:00:00:00**, or ND packets with **nd.sll == 00:00:00:00:00:00**. These match ARP or ND packets whose inner MAC field is zero (i.e., the optional link-layer address option is absent). The action looks up

only **eth.src**: **reg0[11] = lookup_fdb(inport, eth.src);** and sets **reg0[22] = 1** to mark the inner MAC lookup as satisfied (there is no inner MAC to learn).

- Priority 110 flows for ARP packets (matching **arp**), ND Neighbor Advertisements (matching **nd_na**), and ND Neighbor Solicitations (matching **nd_ns**). These match packets whose inner MAC field is non-zero (the priority−120 zero-MAC guard flows take precedence when it is zero). The action looks up both **eth.src** and the inner MAC field: **reg0[11] = lookup_fdb(inport, eth.src); reg0[22] = lookup_fdb(inport, inner); next;** where **inner** is **arp.sha** for ARP, **nd.tll** for NA, or **nd.sll** for NS.
- Priority 100 flow with the match **inport ==** *p* and action **reg0[11] = lookup_fdb(inport, eth.src); next;**

- For each such localnet logical port *p* following flow is added.
  - Priority 100 flow with the match **inport ==** *p* and action **flags.localnet = 1; reg0[11] = lookup_fdb(inport, eth.src); next;**
- For each localnet logical port *p*, a priority−50 fallback flow is added with the match **inport ==** *p* and action **flags.localnet = 1; next;**. This marks traffic arriving from localnet ports so that downstream stages (e.g., ARP/ND responder) can condition their behavior. When FDB learning is enabled on the localnet port, the priority−100 flow described above already sets **flags.localnet**, so this priority−50 flow only takes effect when FDB learning is not configured.
- One priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 4: Learn MAC of 'unknown' ports.

This table learns the MAC addresses seen on the VIF or 'switch' logical ports with'unknown' address set (note: 'switch' ports have implicit 'unknown' addresses) as well as on localnet ports with localnet_learn_fdb option set if the **lookup_fdb** action returned false in the previous table. For localnet ports (with flags.localnet = 1), lookup_fdb returns true if (port, mac) is found or if a mac is found for a port of type vif.

- For each such VIF logical port *p* with 'unknown' address set and localnet port following flow is added.
  - Priority 120 flows matching the same zero-MAC conditions as the corresponding lookup flows (**arp.sha == 0**, **nd.tll == 0**, or **nd.sll == 0**). Guarded by **reg0[11] == 0**, these learn only **eth.src** via **put_fdb(inport, eth.src); next;**.
  - Priority 110 flows matching ARP, ND NA, or ND NS packets. Guarded by **reg0[11] == 0 || reg0[22] == 0** (i.e., either the outer or the inner MAC is unknown), these learn both **eth.src** and the inner MAC via **put_fdb(inport, eth.src); put_fdb(inport, inner); next;** where **inner** is **arp.sha**, **nd.tll**, or **nd.sll** respectively.
  - Priority 100 flow with the match **inport ==** *p* **&& reg0[11] == 0** and action **put_fdb(inport, eth.src); next;** which stores the **port−mac** in the mac learning table of the logical switch datapath and advances the packet to the next table.
- One priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 5: **from−lport** *Pre-ACLs*

This table prepares flows for possible stateful ACL processing in ingress table **ACLs**. It contains a priority−0 flow that simply moves traffic to the next table. If stateful ACLs are used in the logical datapath, a priority−100 flow is added that sets a hint (with **reg0[0] = 1; next;**) for table **Pre−stateful** to send IP packets to the connection tracker before eventually advancing to ingress table **ACLs**. If special ports such as route ports or localnet ports can't use ct(), a priority−110 flow is added to skip over stateful ACLs. This priority−110 flow is not addd for router ports if the option enable_router_port_acl is set to true in

options:enable_router_port_acl column of **Logical_Switch_Port**. Multicast, IPv6 Neighbor Discovery and MLD traffic also skips stateful ACLs. For "allow-stateless" ACLs, a flow is added to bypass setting the hint for connection tracker processing when there are stateful ACLs or LB rules; **REGBIT_ACL_STATELESS** is set for traffic matching stateless ACL flows.

This table also has a priority−110 flow with the match **eth.dst ==** *E* for all logical switch datapaths to move traffic to the next table. Where *E* is the service monitor mac defined in the **options:svc_monitor_mac** column of **NB_Global** table.

Ingress Table 6: Pre-LB

This table prepares flows for possible stateful load balancing processing in ingress table **LB** and **Stateful**. It contains a priority−0 flow that simply moves traffic to the next table. Moreover it contains two priority−110 flows to move multicast, IPv6 Neighbor Discovery and MLD traffic to the next table. It also contains two priority−110 flows to move stateless traffic, i.e traffic for which **REGBIT_ACL_STATELESS** is set, to the next table. If load balancing rules with virtual IP addresses (and ports) are configured in **OVN_Northbound** database for a logical switch datapath, a priority−100 flow is added with the match **ip** to match on IP packets and sets the action **reg0[2] = 1; next;** to act as a hint for table **Pre−stateful** to send IP packets to the connection tracker for packet de-fragmentation (and to possibly do DNAT for already established load balanced traffic) before eventually advancing to ingress table **Stateful**. If controller_event has been enabled and load balancing rules with empty backends have been added in **OVN_Northbound**, a 130 flow is added to trigger ovn-controller events whenever the chassis receives a packet for that particular VIP. If **event−elb** meter has been previously created, it will be associated to the empty_lb logical flow

Prior to **OVN 20.09** we were setting the **reg0[0] = 1** only if the IP destination matches the load balancer VIP. However this had few issues cases where a logical switch doesn't have any ACLs with **allow−related** action. To understand the issue lets a take a TCP load balancer - **10.0.0.10:80=10.0.0.3:80**. If a logical port - p1 with IP - 10.0.0.5 opens a TCP connection with the VIP - 10.0.0.10, then the packet in the ingress pipeline of 'p1' is sent to the p1's conntrack zone id and the packet is load balanced to the backend - 10.0.0.3. For the reply packet from the backend lport, it is not sent to the conntrack of backend lport's zone id. This is fine as long as the packet is valid. Suppose the backend lport sends an invalid TCP packet (like incorrect sequence number), the packet gets delivered to the lport 'p1' without unDNATing the packet to the VIP - 10.0.0.10. And this causes the connection to be reset by the lport p1's VIF.

We can't fix this issue by adding a logical flow to drop ct.inv packets in the egress pipeline since it will drop all other connections not destined to the load balancers. To fix this issue, we send all the packets to the conntrack in the ingress pipeline if a load balancer is configured. We can now add a lflow to drop ct.inv packets.

This table also has priority−120 flows that punt all IGMP/MLD packets to **ovn−controller** if the switch is an interconnect switch with multicast snooping enabled.

This table also has a priority−110 flow with the match **eth.dst ==** *E* for all logical switch datapaths to move traffic to the next table. Where *E* is the service monitor mac defined in the **options:svc_monitor_mac** column of **NB_Global** table.

This table also has a priority−110 flow with the match **inport ==** *I* for all logical switch datapaths to move traffic to the next table. Where *I* is the peer of a logical router port. This flow is added to skip the connection tracking of packets which enter from logical router datapath to logical switch datapath.

Ingress Table 7: Pre-stateful

This table prepares flows for all possible stateful processing in next tables. It contains a priority−0 flow that simply moves traffic to the next table.

- Priority−120 flows that send the packets to connection tracker using **ct_lb_mark;** as the action so that the already established traffic destined to the load balancer VIP gets DNATted. These flows match each VIPs IP and port. For IPv4 traffic the flows also load the original destination IP and transport port in registers **reg1** and **reg2**. For IPv6 traffic the flows also load the original destination IP and transport port in registers **xxreg1** and **reg2**.

- A priority−110 flow sends the packets that don't match the above flows to connection tracker based on a hint provided by the previous tables (with a match for **reg0[2] == 1**) by using the **ct_lb_mark;** action.
- A priority−105 added enabled when enable-stateless-acl-with-lb and send all packet directed to VIP that don't match the above flows to connection tracker.
- A priority−100 flow sends the packets to connection tracker based on a hint provided by the previous tables (with a match for **reg0[0] == 1**) by using the **ct_next;** action.

Ingress Table 8: **from−lport** *ACL hints*

This table consists of logical flows that set hints (**reg0** bits) to be used in the next stage, in the ACL processing table, if stateful ACLs or load balancers are configured. Multiple hints can be set for the same packet. The possible hints are:

- **reg0[7]**: the packet might match an **allow−related** ACL and might have to commit the connection to conntrack.
- **reg0[8]**: the packet might match an **allow−related** ACL but there will be no need to commit the connection to conntrack because it already exists.
- **reg0[9]**: the packet might match a **drop/reject**.
- **reg0[10]**: the packet might match a **drop/reject** ACL but the connection was previously allowed so it might have to be committed again with **ct_label=1/1**.

The table contains the following flows:

- A priority−65535 flow to advance to the next table if the logical switch has **no** ACLs configured, otherwise a priority−0 flow to advance to the next table.
- A priority−7 flow that matches on packets that initiate a new session. This flow sets **reg0[7]** and **reg0[9]** and then advances to the next table.
- A priority−6 flow that matches on packets that are in the request direction of an already existing session that has been marked as blocked. This flow sets **reg0[7]** and **reg0[9]** and then advances to the next table.
- A priority−5 flow that matches untracked packets. This flow sets **reg0[8]** and **reg0[9]** and then advances to the next table.
- A priority−4 flow that matches on packets that are in the request direction of an already existing session that has not been marked as blocked. This flow sets **reg0[8]** and **reg0[10]** and then advances to the next table.
- A priority−3 flow that matches on packets that are in not part of established sessions. This flow sets **reg0[9]** and then advances to the next table.
- A priority−2 flow that matches on packets that are part of an established session that has been marked as blocked. This flow sets **reg0[9]** and then advances to the next table.
- A priority−1 flow that matches on packets that are part of an established session that has not been marked as blocked. This flow sets **reg0[10]** and then advances to the next table.

Ingress table 9: **from−lport** *ACL evaluation before LB*

Logical flows in this table closely reproduce those in the **ACL** table in the **OVN_Northbound** database for the **from−lport** direction without the option **apply−after−lb** set or set to **false**. The **priority** values from the **ACL** table have a limited range and have 1000 added to them to leave room for OVN default flows at both higher and lower priorities.

- This table is responsible for evaluating ACLs, and setting a register bit to indicate whether the ACL decided to allow, drop, or reject the traffic. The allow bit is **reg8[16]**. The drop bit is **reg8[17]**. All flows in this table will advance the packet to the next table, where the bits from before are evaluated to determine what to do with the packet. Any

flows in this table that intend for the packet to pass will set **reg8[16]** to 1, even if an ACL with an allow-type action was not matched. This lets the next table know to allow the traffic to pass. These bits will be referred to as the "allow", "drop", and "reject" bits in the upcoming paragraphs.

- If the **tier** column has been configured on the ACL, then OVN will also match the current tier counter against the configured ACL tier. OVN keeps count of the current tier in **reg8[30..31]**.
- **allow** ACLs translate into logical flows that set the allow bit to 1 and advance the packet to the next table. If there are any stateful ACLs on this datapath, then **allow** ACLs set the allow bit to one and in addition perform **ct_commit;** (which acts as a hint for future tables to commit the connection to conntrack). In case the **ACL** has a label then **reg3** is loaded with the label value and **reg0[13]** bit is set to 1 (which acts as a hint for the next tables to commit the label to conntrack).
- **allow−related** ACLs translate into logical flows that set the allow bit and additionally have **ct_commit { ct_label=0/1; }; next;** actions for new connections and **reg0[1] = 1; next;** for existing connections. In case the **ACL** has a label then **reg3** is loaded with the label value and **reg0[13]** bit is set to 1 (which acts as a hint for the next tables to commit the label to conntrack).
- For **allow** and **allow−related** ACL, an additonal set of registers get set in case the ACL has the column **network_function_group** set to the *id* of one of the entities in **Network_Function_Group** table. The *id* is an internally generated unique identifier for a **Network_Function_Group** entity. The flow sets **reg8[21] = 1** (to indicate need for packet redirection), **reg8[22] = 1** (to indicate this is a request packet) and **reg0[22..29] =** *id*. These registers are later used in the **Network Function** table.
- **allow−stateless** ACLs translate into logical flows that set the allow bit and advance to the next table.
- **reject** ACLs translate into logical flows with that set the reject bit and advance to the next table.
- **pass** ACLs translate into logical flows that do not set the allow, drop, or reject bit and advance to the next table.
- Other ACLs set the drop bit and advance to the next table for new or untracked connections. For known connections, they set the drop bit, as well as running the **ct_commit { ct_label=1/1; };** action. Setting **ct_label** marks a connection as one that was previously allowed, but should no longer be allowed due to a policy change.

This table contains a priority−65535 flow to set the allow bit and advance to the next table if the logical switch has **no** ACLs configured, otherwise a priority−0 flow to advance to the next table is added. This flow does not set the allow bit, so that the next table can decide whether to allow or drop the packet based on the value of the **options:default_acl_drop** column of the **NB_Global** table.

A priority−65532 flow is added that sets the allow bit for IPv6 Neighbor solicitation, Neighbor discover, Router solicitation, Router advertisement and MLD packets regardless of other ACLs defined.

If the logical datapath has a stateful ACL or a load balancer with VIP configured, the following flows will also be added:

- If **options:default_acl_drop** column of **NB_Global** is **false** or not set, a priority−1 flow that sets the hint to commit IP traffic that is not part of established sessions to the connection tracker (with action **reg0[1] = 1; next;**). This is needed for the default allow policy because, while the initiator's direction may not have any stateful rules, the server's may and then its return traffic would not be known and marked as invalid.

- A priority−1 flow that sets the allow bit and sets the hint to commit IP traffic to the connection tracker (with action **reg0[1] = 1; next;**). This is needed for the default allow policy because, while the initiator's direction may not have any stateful rules, the server's may and then its return traffic would not be known and marked as invalid.
- A priority−65532 flow that sets the allow bit for any traffic in the reply direction for a connection that has been committed to the connection tracker (i.e., established flows), as long as the committed flow does not have **ct_mark.blocked** set. We only handle traffic in the reply direction here because we want all packets going in the request direction to still go through the flows that implement the currently defined policy based on ACLs. If a connection is no longer allowed by policy, **ct_mark.blocked** will get set and packets in the reply direction will no longer be allowed, either. This flow also clears the register bits **reg0[9]** and **reg0[10]** and sets register bit **reg0[17]**. If ACL logging and logging of related packets is enabled, then a companion priority−65533 flow will be installed that accomplishes the same thing but also logs the traffic.
- The priority−65532 flows that allow response and related traffic, also set **reg8[21] = ct_label.nf**, which gets checked in the **Network Function** table.
- A priority−65532 flow that sets the allow bit for any traffic that is considered related to a committed flow in the connection tracker (e.g., an ICMP Port Unreachable from a non-listening UDP port), as long as the committed flow does not have **ct_mark.blocked** set. This flow also applies NAT to the related traffic so that ICMP headers and the inner packet have correct addresses. If ACL logging and logging of related packets is enabled, then a companion priority−65533 flow will be installed that accomplishes the same thing but also logs the traffic.
- A priority−65532 flow that sets the drop bit for all traffic marked by the connection tracker as invalid.
- A priority−65532 flow that sets the drop bit for all traffic in the reply direction with **ct_mark.blocked** set meaning that the connection should no longer be allowed due to a policy change. Packets in the request direction are skipped here to let a newly created ACL re-allow this connection.

If the logical datapath has any ACL or a load balancer with VIP configured, the following flow will also be added:

- A priority 34000 logical flow is added for each logical switch datapath with the match **eth.dst =** *E* to allow the service monitor reply packet destined to **ovn−controller** that sets the allow bit, where *E* is the service monitor mac defined in the **options:svc_monitor_mac** column of **NB_Global** table.

Ingress Table 10: **from−lport** *ACL sampling*

Logical flows in this table sample traffic matched by **from−lport** ACLs with sampling enabled.

- If no ACLs have sampling enabled, then a priority 0 flow is installed that matches everything and advances to the next table.
- For each ACL with sample_new configured a priority 1100 flow is installed that matches on the saved observation_point_id value. This flow generates a **sample()** action and then advances the packet to the next table.
- For each ACL with sample_est configured a priority 1200 flow is installed that matches on the saved observation_point_id value for established traffic in the original direction. This flow generates a **sample()** action and then advances the packet to the next table.
- For each ACL with sample_est configured a priority 1200 flow is installed that matches on the saved observation_point_id value for established traffic in the reply direction. This flow generates a **sample()** action and then advances the packet to the next table. Note: this flow is installed in the opposite pipeline (in the ingress pipeline for ACLs applied in

the egress direction and in the egress pipeline for ACLs applied in the ingress direction).

Ingress Table 11: **from−lport** *ACL action*

Logical flows in this table decide how to proceed based on the values of the allow, drop, and reject bits that may have been set in the previous table.

- If no ACLs are configured, then a priority 0 flow is installed that matches everything and advances to the next table.
- A priority 1000 flow is installed that will advance the packet to the next table if the allow bit is set.
- A priority 1000 flow is installed that will run the **drop;** action if the drop bit is set.
- A priority 1000 flow is installed that will run the **tcp_reset { output <−> inport; next(pipeline=egress,table=5);}** action for TCP connections,**icmp4/icmp6** action for UDP connections, and **sctp_abort {output <−%gt; inport; next(pipeline=egress,table=5);}** action for SCTP associations.
- If any ACLs have tiers configured on them, then three priority 500 flows are installed. If the current tier counter is 0, 1, or 2, then the current tier counter is incremented by one and the packet is sent back to the previous table for re-evaluation.

Ingress Table 12: **from−lport** *QoS*

Logical flows in this table closely reproduce those in the **QoS** table with the **action** or **bandwidth** column set in the **OVN_Northbound** database for the **from−lport** direction.

- For every qos_rules entry in a logical switch with DSCP marking, packet marking or metering enabled a flow will be added at the priority mentioned in the QoS table.
- One priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 13: Connection Tracking Field Extraction

This table extracts connection tracking fields for new connections to be used by subsequent load balancing stages.

- A priority−100 flow matches **ct.new && ip** and extracts connection tracking protocol and destination port information into registers **reg1[16..23]** (protocol) and **reg1[0..15]** (destination port) using the actions **reg1[16..23] = ct_proto(); reg1[0..15] = ct_tp_dst(); next;**.
- A priority−0 flow matches all packets and advances to the next table.

Ingress Table 14: Load balancing affinity check

Load balancing affinity check table contains the following logical flows:

- For all the configured load balancing rules for a switch in **OVN_Northbound** database where a positive affinity timeout is specified in **options** column, that includes a L4 port *PORT* of protocol *P* and IP address *VIP*, a priority−100 flow is added. For IPv4 *VIPs*, the flow matches **ct.new && ip && ip4.dst ==** *VIP* **&& reg1[16..23] ==** *PROTO_NUM* **&& reg1[0..15] ==** *PORT*. For IPv6 *VIPs*, the flow matches **ct.new && ip && ip6.dst ==** *VIP* **&& reg1[16..23] ==** *PROTO_NUM* **&& reg1[0..15] ==** *PORT*. The flow's action is **reg9[6] = chk_lb_aff(); next;**.
- A priority 0 flow is added which matches on all packets and applies the action **next;**.

Ingress Table 15: LB

- For all the configured load balancing rules for a switch in **OVN_Northbound** database where a positive affinity timeout is specified in **options** column, that includes a L4 port *PORT* of protocol *P* and IP address *VIP*, a priority−150 flow is added. For IPv4 *VIPs*, the flow matches **reg9[6] == 1 && ct.new && ip && ip4.dst ==** *VIP* **&&** *P***.dst ==** *PORT* . For IPv6 *VIPs*, the flow matches **reg9[6] == 1 && ct.new && ip && ip6.dst ==** *VIP*

&& *P* && *P*.dst == *PORT*. The flow's action is **ct_lb_mark(***args***)**, where *args* contains comma separated IP addresses (and optional port numbers) to load balance to. The address family of the IP addresses of *args* is the same as the address family of *VIP*.

- For all the configured load balancing rules for a switch in **OVN_Northbound** database that includes a L4 port *PORT* of protocol *P* and IP address *VIP*, a priority−120 flow is added. For IPv4 *VIPs* , the flow matches **ct.new && ip && ip4.dst ==** *VIP* **&& reg1[16..23] ==** *PROTO_NUM* **&& reg1[0..15] ==** *PORT*. For IPv6 *VIPs*, the flow matches **ct.new && ip && ip6.dst ==** *VIP* **&& reg1[16..23] ==** *PROTO_NUM* **&& reg1[0..15] ==** *PORT*. The flow's action is **ct_lb_mark(***args***)** , where *args* contains comma separated IP addresses (and optional port numbers) to load balance to. The address family of the IP addresses of *args* is the same as the address family of *VIP*. If health check is enabled, then *args* will only contain those endpoints whose service monitor status entry in **OVN_Southbound** db is either **online** or empty. For IPv4 traffic the flow also loads the original destination IP and transport port in registers **reg1** and **reg2**. For IPv6 traffic the flow also loads the original destination IP and transport port in registers **xxreg1** and **reg2**. The above flow is created even if the load balancer is attached to a logical router connected to the current logical switch and the **install_ls_lb_from_router** variable in **options** is set to true.
- For all the configured load balancing rules for a switch in **OVN_Northbound** database that includes just an IP address *VIP* to match on, OVN adds a priority−110 flow. For IPv4 *VIPs*, the flow matches **ct.new && ip && ip4.dst ==** *VIP*. For IPv6 *VIPs*, the flow matches **ct.new && ip && ip6.dst ==** *VIP*. The action on this flow is **ct_lb_mark(***args***)**, where *args* contains comma separated IP addresses of the same address family as *VIP*. For IPv4 traffic the flow also loads the original destination IP and transport port in registers **reg1** and **reg2**. For IPv6 traffic the flow also loads the original destination IP and transport port in registers **xxreg1** and **reg2**. The above flow is created even if the load balancer is attached to a logical router connected to the current logical switch and the **install_ls_lb_from_router** variable in **options** is set to true.
- If the load balancer is created with **−−reject** option and it has no active backends, a TCP reset segment (for tcp) or an ICMP port unreachable packet (for all other kind of traffic) will be sent whenever an incoming packet is received for this load-balancer. Please note using **−−reject** option will disable empty_lb SB controller event for this load balancer.

Ingress Table 16: Load balancing affinity learn

Load balancing affinity learn table contains the following logical flows:

- For all the configured load balancing rules for a switch in **OVN_Northbound** database where a positive affinity timeout *T* is specified in **options** column, that includes a L4 port *PORT* of protocol *P* and IP address *VIP*, a priority−100 flow is added. For IPv4 *VIPs*, the flow matches **reg9[6] == 0 && ct.new && ip && ip4.dst ==** *VIP* **&& *P*.dst ==** *PORT*. For IPv6 *VIPs*, the flow matches **ct.new && ip && ip6.dst ==** *VIP* **&& *P* && *P*.dst ==** *PORT* . The flow's action is **commit_lb_aff(vip =** *VIP***:***PORT***, backend =** *backend ip***:** *backend port***, proto =** *P***, timeout =** *T***);** .
- A priority 0 flow is added which matches on all packets and applies the action **next;**.

Ingress Table 17: Pre-Hairpin

- If the logical switch has load balancer(s) configured, then a priority−100 flow is added with the match **ip && ct.trk** to check if the packet needs to be hairpinned (if after load balancing the destination IP matches the source IP) or not by executing the actions **reg0[6] = chk_lb_hairpin();** and **reg0[12] = chk_lb_hairpin_reply();** and advances the packet to the next table.

- A priority−0 flow that simply moves traffic to the next table.

Ingress Table 18: Nat-Hairpin

- If the logical switch has load balancer(s) configured, then a priority−100 flow is added with the match **ip && ct.new && ct.trk && reg0[6] == 1** which hairpins the traffic by NATting source IP to the load balancer VIP by executing the action **ct_snat_to_vip** and advances the packet to the next table.
- If the logical switch has load balancer(s) configured, then a priority−100 flow is added with the match **ip && ct.est && ct.trk && reg0[6] == 1** which hairpins the traffic by NATting source IP to the load balancer VIP by executing the action **ct_snat** and advances the packet to the next table.
- If the logical switch has load balancer(s) configured, then a priority−90 flow is added with the match **ip && reg0[12] == 1** which matches on the replies of hairpinned traffic (i.e., destination IP is VIP, source IP is the backend IP and source L4 port is backend port for L4 load balancers) and executes **ct_snat** and advances the packet to the next table.
- A priority−0 flow that simply moves traffic to the next table.

Ingress Table 19: Hairpin

- If logical switch has attached logical switch port of *vtep* type, then for each distributed gateway router port *RP* attached to this logical switch and has chassis redirect port *cr-RP*, a priority−2000 flow is added with the match .IP
  reg0[14] == 1 && is_chassis_resident(*cr-RP***)**

  and action **next;**.

  reg0[14] register bit is set in the ingress L2 port security check table for traffic received from HW VTEP (ramp) ports.
- If logical switch has attached logical switch port of *vtep* type, then a priority−1000 flow that matches on **reg0[14]** register bit for the traffic received from HW VTEP (ramp) ports. This traffic is passed to ingress table ls_in_l2_lkup.
- A priority−1 flow that hairpins traffic matched by non-default flows in the Pre-Hairpin table. Hairpinning is done at L2, Ethernet addresses are swapped and the packets are looped back on the input port.
- A priority−0 flow that simply moves traffic to the next table.

Ingress table 20: **from−lport** *ACL evaluation after LB*

Logical flows in this table closely reproduce those in the **ACL eval** table in the **OVN_Northbound** database for the **from−lport** direction with the option **apply−after−lb** set to **true**. The **priority** values from the **ACL** table have a limited range and have 1000 added to them to leave room for OVN default flows at both higher and lower priorities. The flows in this table indicate the ACL verdict by setting **reg8[16]** for **allow−type** ACLs, **reg8[17]** for **drop** ACLs, and **reg8[17]** for **reject** ACLs, and then advancing the packet to the next table. These will be reffered to as the allow bit, drop bit, and reject bit throughout the documentation for this table and the next one.

Like with ACLs that are evaluated before load balancers, if the ACL is configured with a tier value, then the current tier counter, supplied in reg8[30..31] is matched against the ACL's configured tier in addition to the ACL's match.

- **allow** apply-after-lb ACLs translate into logical flows that set the allow bit. If there are any stateful ACLs (including both before-lb and after-lb ACLs) on this datapath, then **allow** ACLs also run **ct_commit; next;** (which acts as a hint for an upcoming table to commit the connection to conntrack). In case the **ACL** has a label then **reg3** is loaded with the label value and **reg0[13]** bit is set to 1 (which acts as a hint for the next tables to commit the label to conntrack).

- **allow−related** apply-after-lb ACLs translate into logical flows that set the allow bit and run the **ct_commit {ct_label=0/1; }; next;** actions for new connections and **reg0[1] = 1; next;** for existing connections. In case the **ACL** has a label then **reg3** is loaded with the label value and **reg0[13]** bit is set to 1 (which acts as a hint for the next tables to commit the label to conntrack).
- **allow−stateless** apply-after-lb ACLs translate into logical flows that set the allow bit and advance to the next table.
- **reject** apply-after-lb ACLs translate into logical flows that set the reject bit and advance to the next table.
- **pass** apply-after-lb ACLs translate into logical flows that do not set the allow, drop, or reject bit and advance to the next table.
- Other apply-after-lb ACLs set the drop bit for new or untracked connections and **ct_commit { ct_label=1/1; }** for known connections. Setting **ct_label** marks a connection as one that was previously allowed, but should no longer be allowed due to a policy change.
- One priority−65532 flow matching packets with **reg0[17]** set (either replies to existing sessions or traffic related to existing sessions) and allows these by setting the allow bit and advancing to the next table.
- One priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 21: **from−lport** *ACL sampling after LB*

Logical flows in this table sample traffic matched by **from−lport** ACLs (evaluation after LB) with sampling enabled.

- If no ACLs have sampling enabled, then a priority 0 flow is installed that matches everything and advances to the next table.
- For each ACL with sample_new configured a priority 1100 flow is installed that matches on the saved observation_point_id value. This flow generates a **sample()** action and then advances the packet to the next table.
- For each ACL with sample_est configured a priority 1200 flow is installed that matches on the saved observation_point_id value for established traffic in the original direction. This flow generates a **sample()** action and then advances the packet to the next table.
- For each ACL with sample_est configured a priority 1200 flow is installed that matches on the saved observation_point_id value for established traffic in the reply direction. This flow generates a **sample()** action and then advances the packet to the next table. Note: this flow is installed in the opposite pipeline (in the ingress pipeline for ACLs applied in the egress direction and in the egress pipeline for ACLs applied in the ingress direction).

Ingress Table 22: **from−lport** *ACL action after LB*

Logical flows in this table decide how to proceed based on the values of the allow, drop, and reject bits that may have been set in the previous table.

- If no ACLs are configured, then a priority 0 flow is installed that matches everything and advances to the next table.
- A priority 1000 flow is installed that will advance the packet to the next table if the allow bit is set.
- A priority 1000 flow is installed that will run the **drop;** action if the drop bit is set.
- A priority 1000 flow is installed that will run the **tcp_reset { output <−> inport; next(pipeline=egress,table=5);}** action for TCP connections,**icmp4/icmp6** action for UDP connections, and **sctp_abort {output <−%gt; inport; next(pipeline=egress,table=5);}** action for SCTP associations.

- If any ACLs have tiers configured on them, then three priority 500 flows are installed. If the current tier counter is 0, 1, or 2, then the current tier counter is incremented by one and the packet is sent back to the previous table for re-evaluation.

Ingress Table 23: Pre Network Function

This stage selects the active network function from a **Network_Function_Group** based on the network function group ID set by the ACL eval stage earlier. This stage is applicable for request packets of **from−lport** ACLs (**reg8[22] == 1**). Response packets for **to−lport** ACLs bypass this stage and use **ct_label.nf_id** directly in the Network Function table.

A network function group can contain one or more network functions. Health monitoring is performed by sending datapath probes as per parameters defined in **Network_Function_Health_Check**. This stage selects one of the healthy network functions. If none are healthy, the behavior follows the **fallback** column configured in the **Network_Function_Group** table. If health monitoring is not configured, any one from the group is selected.

When a request packet matches a **from−lport** ACL with **network_function_group** set, the ACL eval stage sets **reg8[21] = 1** (NF enabled), **reg8[22] = 1** (request direction), and **reg0[22..29]** to the network function group ID. This table then selects the active network function from the group and overwrites **reg0[22..29]** with the specific **id** of a **Network_Function** table entry. The subsequent Network Function table uses this NF ID to redirect packets to the appropriate network function port. In the future, this stage will be extended to support network function load balancing.

- For each network_function_group *id* with an active network function, a priority−99 flow matches **reg8[21] == 1 && reg8[22] == 1 && reg0[22..29] ==** *id* and sets **reg0[22..29] =** *nf_id***; next;** where *nf_id* is the ID of the active network function. This prepares request packets that matched a **from−lport** ACL with network_function_group for redirection in the subsequent Network Function table.
- For each network function group with *id* that has **fallback** set to **fail−open**, a priority−10 flow matches **reg8[21] == 1 && reg8[22] == 1 && reg0[22..29] ==** *id* and sets **reg8[21] = 0; reg0[22..29] = 0; next;**. This clears both the NF enabled bit and the NF group ID, allowing packets to continue processing through the pipeline without network function redirection when no active network function is available (fail-open behavior).
- A priority−1 flow matches **reg8[21] == 1 && reg8[22] == 1** and sets **reg0[22..29] = 0; next;**. This is a catch-all flow for network function groups with **fallback** set to **fail−close** (or default) when no active network function is available. It clears only the NF group ID, leaving the NF enabled bit set. These packets will be dropped by the priority−1 drop rule in the subsequent Network Function table (fail-close behavior).
- A priority−0 flow that simply moves traffic to the next table.

Ingress Table 24: Stateful

- A priority 100 flow is added which commits the packet to the conntrack and sets the most significant 32-bits of **ct_label** with the **reg3** value based on the hint provided by previous tables (with a match for **reg0[1] == 1 && reg0[13] == 1**). This is used by the **ACLs** with label to commit the label value to conntrack.
- For **ACLs** without label, a second priority−100 flow commits packets to connection tracker using **ct_commit; next;** action based on a hint provided by the previous tables (with a match for **reg0[1] == 1 && reg0[13] == 0**).
- Corresponding to each of the two priority 100 flows above, a priority 110 flow is added, which has the following extra match and action, but otherwise identical to the priority 100 flow. Match: **reg8[21] == 1** (packet matched an ACL with **network_function_group** set) Action: **ct_label.nf = 1; ct_label.nf_id = reg0[22..29];** This is to commit the network_function information in conntrack so that the response and related packets can be redirected to it as well.

- A priority−0 flow that simply moves traffic to the next table.

Ingress Table 25: Network Function

This table implements packet redirection to network functions. When a packet matches an ACL with **network_function_group** column set to the **id** of a **Network_Function_Group** table entry, the ACL eval stage sets **reg8[21] = 1** (NF enabled), **reg8[22] = 1** (request direction), and **reg0[22..29]** to the network function group ID. The Pre Network Function stage then selects the active network function from the group and overwrites **reg0[22..29]** with the specific **id** of a **Network_Function** table entry. This table uses that NF ID to redirect packets to the appropriate network function port.

This table handles request packets for **from−lport** ACLs and response packets for **to−lport** ACLs. For **from−lport** ACLs, request packets are redirected to the network function's **inport**, and corresponding response/related packets are handled in the egress pipeline. For **to−lport** ACLs, request packets are handled in the egress pipeline, but corresponding response/related packets for those flows are redirected here using the network function ID stored in **ct_label.nf_id** during request processing.

If the network function ports are not present on this logical switch, their child ports (if any) are used. In the statements below, network function ports refer to either the parent or child ports as applicable to this logical switch.

- For each network_function port *P*, a priority−100 flow is added that matches **inport ==** *P* and advances packets to the next table. Thus packets coming from network function are not subject to redirection. This flow also sets **reg5[16..31] = ct_label.tun_if_id**. This is used for tunneling packet to originating host in case of cross host traffic redirection for VLAN subnet. This ct_label field stores the openflow tunnel interface id of the originating host for this connection and gets populated in egress **Stateful** table.
- For each active network function with *id* that is referenced in a network function group, a priority−99 flow matches **reg8[21] == 1 && reg8[22] == 1 && reg0[22..29] ==** *id* and sets **outport=***P***; output;** where *P* is the **inport** of that network function. This redirects request packets for flows matching **from−lport** ACLs with network_function_group to the specific network function selected by the Pre Network Function stage.
- For each active network function with *id* that is referenced in a network function group, a priority−99 rule matches **reg8[21] == 1 && reg8[22] == 0 && ct_label.nf_id ==** *id* and takes identical action as above. This redirects response and related packets for **to−lport** ACLs to the same network function that handled the request, using the NF ID stored in the connection tracking label.
- In each of the above cases, when the same packet comes out unchanged through the other port of the network_function, it would match the priority 100 flow and be forwarded to the next table.
- One priority−100 rule to skip redirection of multicast packets that hit a network_function ACL. Match on **reg8[21] == 1 && eth.mcast** and action is to advance to the next table.
- One priority−1 rule that checks **reg8[21] == 1**, and drops such packets. This is to address the case where a packet hit an ACL with network function but the network function does not have ports or child ports on this logical switch.
- One priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 26: ARP/ND responder

This table implements ARP/ND responder in a logical switch for known IPs. The advantage of the ARP responder flow is to limit ARP broadcasts by locally responding to ARP requests without the need to send to other hypervisors. One common case is when the inport is a logical port associated with a VIF and the broadcast is responded to on the local hypervisor rather than broadcast across the whole network and responded to by the destination VM. This behavior is proxy ARP.

ARP requests arrive from VMs from a logical switch inport of type default. For this case, the logical switch proxy ARP rules can be for other VMs or logical router ports. Logical switch proxy ARP rules may be

programmed both for mac binding of IP addresses on other logical switch VIF ports (which are of the default logical switch port type, representing connectivity to VMs or containers), and for mac binding of IP addresses on logical switch router type ports, representing their logical router port peers. In order to support proxy ARP for logical router ports, an IP address must be configured on the logical switch router type port, with the same value as the peer logical router port. The configured MAC addresses must match as well. When a VM sends an ARP request for a distributed logical router port and if the peer router type port of the attached logical switch does not have an IP address configured, the ARP request will be broadcast on the logical switch. One of the copies of the ARP request will go through the logical switch router type port to the logical router datapath, where the logical router ARP responder will generate a reply. The MAC binding of a distributed logical router, once learned by an associated VM, is used for all that VM's communication needing routing. Hence, the action of a VM re-arping for the mac binding of the logical router port should be rare.

Logical switch ARP responder proxy ARP rules can also be hit when receiving ARP requests externally on a L2 gateway port. In this case, the hypervisor acting as an L2 gateway, responds to the ARP request on behalf of a destination VM.

ARP/ND requests received from **localnet** logical inports do hit the ARP/ND responder, but the response is limited to the chassis that hosts the target VIF. This is achieved by adding a **flags.localnet** check to the priority−50 reply flows (see below): when the request arrives from a localnet port (**flags.localnet == 1**), only the chassis on which the target port is resident will reply. When the request arrives from a non-localnet port (**flags.localnet == 0**), the response is unconditional, preserving VIF-to-VIF proxy ARP/ND behavior. It contains these logical flows:

- If packet was received from HW VTEP (ramp switch), and this packet is ARP or Neighbor Solicitation, such packet is passed to next table with max proirity. ARP/ND requests from HW VTEP must be handled in logical router ingress pipeline.
- ARP/ND requests received on an inport of type **router** are not expected to hit any logical switch ARP responder flows. However, no skip flows are installed for these packets, as there would be some additional flow cost for this and the value appears limited.
- If inport **V** is of type **virtual** adds a priority−100 logical flows for each *P* configured in the **options:virtual-parents** column with the match

  inport == *P* **&& && ((arp.op == 1 && arp.spa ==** *VIP* **&& arp.tpa ==** *VIP*) **|| (arp.op == 2 && arp.spa**
  inport == *P* **&& && ((nd_ns && ip6.dst ==** *{VIP, NS_MULTICAST_ADDR}* **&& nd.target ==** *VIP*) **|| (n**

  and applies the action

  bind_vport(*V***, inport);**

  and advances the packet to the next table.

  Where *VIP* is the virtual ip configured in the column **options:virtual-ip** and NS_MULTICAST_ADDR is solicited-node multicast address corresponding to the VIP.
- Priority−50 flows that match only broadcast ARP requests to each known IPv4 address *A* of every logical switch port, and respond with ARP replies directly with corresponding Ethernet address *E*:

  eth.dst = eth.src;
  eth.src = *E*;
  arp.op = 2; /* ARP reply. */
  arp.tha = arp.sha;
  arp.sha = *E*;
  arp.tpa = arp.spa;
  arp.spa = *A*;
  outport = inport;
  flags.loopback = 1;

output;

On logical switches that have a localnet port, the match for these flows includes an additional condition: **((flags.localnet == 1 && is_chassis_resident(***port***)) || flags.localnet == 0)**. This ensures that when an ARP request arrives from a localnet port, only the chassis hosting the target VIF responds. When the request arrives from a non-localnet port, the response is unconditional, preserving VIF-to-VIF proxy ARP behavior.

These flows are omitted for logical ports (other than router ports or **localport** ports) that are down (unless **ignore_lsp_down** is configured as true in **options** column of **NB_Global** table of the **Northbound** database), for logical ports of type **virtual**, for logical ports with 'unknown' address set, for logical ports with the **options:disable_arp_nd_rsp=true** and for logical ports of a logical switch configured with **other_config:vlan−passthru=true**.

The above ARP responder flows are added for the list of IPv4 addresses if defined in **options:arp_proxy** column of **Logical_Switch_Port** table for logical switch ports of type **router**.

- Priority−50 flows that match IPv6 ND neighbor solicitations to each known IP address *A* (and *A*'s solicited node address) of every logical switch port except of type router, and respond with neighbor advertisements directly with corresponding Ethernet address *E*:

  nd_na {
  eth.src = *E*;
  ip6.src = *A*;
  nd.target = *A*;
  nd.tll = *E*;
  outport = inport;
  flags.loopback = 1;
  output;
  };

  Priority−50 flows that match IPv6 ND neighbor solicitations to each known IP address *A* (and *A*'s solicited node address) of logical switch port of type router, and respond with neighbor advertisements directly with corresponding Ethernet address *E*:

  nd_na_router {
  eth.src = *E*;
  ip6.src = *A*;
  nd.target = *A*;
  nd.tll = *E*;
  outport = inport;
  flags.loopback = 1;
  output;
  };

  On logical switches that have a localnet port, the match for these flows includes an additional condition: **((flags.localnet == 1 && is_chassis_resident(***port***)) || flags.localnet == 0)**. This ensures that when an ND solicitation arrives from a localnet port, only the chassis hosting the target VIF responds. When the solicitation arrives from a non-localnet port, the response is unconditional, preserving VIF-to-VIF proxy ND behavior.

  These flows are omitted for logical ports (other than router ports or **localport** ports) that are down (unless **ignore_lsp_down** is configured as true in **options** column of **NB_Global** table of the **Northbound** database), for logical ports of type **virtual** and for logical ports with 'unknown' address set.

The above NDP responder flows are added for the list of IPv6 addresses if defined in **options:arp_proxy** column of **Logical_Switch_Port** table for logical switch ports of type **router**.

- Priority−100 flows with match criteria similar to the ARP and ND flows above, except that they only match packets from the **inport** that owns the IP addresses in question, with action **next;**. These flows prevent OVN from replying to, for example, an ARP request emitted by a VM for its own IP address. A VM only makes this kind of request to attempt to detect a duplicate IP address assignment, so sending a reply will prevent the VM from accepting the IP address that it owns.

  In place of **next;**, it would be reasonable to use **drop;** for the flows' actions. If everything is working as it is configured, then this would produce equivalent results, since no host should reply to the request. But ARPing for one's own IP address is intended to detect situations where the network is not working as configured, so dropping the request would frustrate that intent.

- For each *SVC_MON_SRC_IP* defined in the value of the **ip_port_mappings:ENDPOINT_IP** column of **Load_Balancer** table, priority−110 logical flow is added with the match **arp.tpa ==** *SVC_MON_SRC_IP* **&& && arp.op == 1** and applies the action

  eth.dst = eth.src;
  eth.src = *E*;
  arp.op = 2; /* ARP reply. */
  arp.tha = arp.sha;
  arp.sha = *E*;
  arp.tpa = arp.spa;
  arp.spa = *A*;
  outport = inport;
  flags.loopback = 1;
  output;

  where *E* is the service monitor source mac defined in the **options:svc_monitor_mac** column in the **NB_Global** table. This mac is used as the source mac in the service monitor packets for the load balancer endpoint IP health checks.

  SVC_MON_SRC_IP is used as the source ip in the service monitor IPv4 packets for the load balancer endpoint IP health checks.

  These flows are required if an ARP request is sent for the IP *SVC_MON_SRC_IP*.

  For IPv6 the similar flow is added with the following action

  nd_na {
  eth.dst = eth.src;
  eth.src = *E*;
  ip6.src = *A*;
  nd.target = *A*;
  nd.tll = *E*;
  outport = inport;
  flags.loopback = 1;
  output;
  };

- For each *VIP* configured in the table **Forwarding_Group** a priority−50 logical flow is added with the match **arp.tpa ==** *vip* **&& && arp.op == 1**
  and applies the action

eth.dst = eth.src;
eth.src = *E*;
arp.op = 2; /* ARP reply. */
arp.tha = arp.sha;
arp.sha = *E*;
arp.tpa = arp.spa;
arp.spa = *A*;
outport = inport;
flags.loopback = 1;
output;

where *E* is the forwarding group's mac defined in the **vmac**.

A is used as either the destination ip for load balancing traffic to child ports or as nexthop to hosts behind the child ports.

These flows are required to respond to an ARP request if an ARP request is sent for the IP *vip*.

- One priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 27: DHCP option processing

This table adds the DHCPv4 options to a DHCPv4 packet from the logical ports configured with IPv4 address(es) and DHCPv4 options, and similarly for DHCPv6 options. This table also adds flows for the logical ports of type **external**.

- A priority−100 logical flow is added for these logical ports which matches the IPv4 packet with **udp.src** = 68 and **udp.dst** = 67 and applies the action **put_dhcp_opts** and advances the packet to the next table.

  reg0[3] = put_dhcp_opts(offer_ip = *ip*, *options...*);
  next;

  For DHCPDISCOVER and DHCPREQUEST, this transforms the packet into a DHCP reply, adds the DHCP offer IP *ip* and options to the packet, and stores 1 into reg0[3]. For other kinds of packets, it just stores 0 into reg0[3]. Either way, it continues to the next table.

- A priority−100 logical flow is added for these logical ports which matches the IPv6 packet with **udp.src** = 546 and **udp.dst** = 547 and applies the action **put_dhcpv6_opts** and advances the packet to the next table.

  reg0[3] = put_dhcpv6_opts(ia_addr = *ip*, *options...*);
  next;

  For DHCPv6 Solicit/Request/Confirm packets, this transforms the packet into a DHCPv6 Advertise/Reply, adds the DHCPv6 offer IP *ip* and options to the packet, and stores 1 into reg0[3]. For other kinds of packets, it just stores 0 into reg0[3]. Either way, it continues to the next table.

- A priority−0 flow that matches all packets to advances to table 16.

Ingress Table 28: DHCP responses

This table implements DHCP responder for the DHCP replies generated by the previous table.

- A priority 100 logical flow is added for the logical ports configured with DHCPv4 options which matches IPv4 packets with **udp.src == 68 && udp.dst == 67 && reg0[3] == 1** and responds back to the **inport** after applying these actions. If **reg0[3]** is set to 1, it means that the action **put_dhcp_opts** was successful.

eth.dst = eth.src;
eth.src = *E*;
ip4.src = *S*;
udp.src = 67;
udp.dst = 68;
outport = *P*;
flags.loopback = 1;
output;

where *E* is the server MAC address and *S* is the server IPv4 address defined in the DHCPv4 options. Note that **ip4.dst** field is handled by **put_dhcp_opts**.

(This terminates ingress packet processing; the packet does not go to the next ingress table.)

- A priority 100 logical flow is added for the logical ports configured with DHCPv6 options which matches IPv6 packets with **udp.src == 546 && udp.dst == 547 && reg0[3] == 1** and responds back to the **inport** after applying these actions. If **reg0[3]** is set to 1, it means that the action **put_dhcpv6_opts** was successful.

  eth.dst = eth.src;
  eth.src = *E*;
  ip6.dst = *A*;
  ip6.src = *S*;
  udp.src = 547;
  udp.dst = 546;
  outport = *P*;
  flags.loopback = 1;
  output;

  where *E* is the server MAC address and *S* is the server IPv6 LLA address generated from the **server_id** defined in the DHCPv6 options and *A* is the IPv6 address defined in the logical port's addresses column.

  (This terminates packet processing; the packet does not go on the next ingress table.)

- A priority−0 flow that matches all packets to advances to table 17.

Ingress Table 29 DNS Lookup

This table looks up and resolves the DNS names to the corresponding configured IP address(es).

- A priority−100 logical flow for each logical switch datapath if it is configured with DNS records, which matches the IPv4 and IPv6 packets with **udp.dst** = 53 and applies the action **dns_lookup** and advances the packet to the next table.

  reg0[4] = dns_lookup(); next;

  For valid DNS packets, this transforms the packet into a DNS reply if the DNS name can be resolved, and stores 1 into reg0[4]. For failed DNS resolution or other kinds of packets, it just stores 0 into reg0[4]. Either way, it continues to the next table.

Ingress Table 30 DNS Responses

This table implements DNS responder for the DNS replies generated by the previous table.

- A priority−100 logical flow for each logical switch datapath if it is configured with DNS records, which matches the IPv4 and IPv6 packets with **udp.dst = 53 && reg0[4] == 1** and responds back to the **inport** after applying these actions. If **reg0[4]** is set to 1, it means that the action **dns_lookup** was successful.

eth.dst <–> eth.src;
ip4.src <–> ip4.dst;
udp.dst = udp.src;
udp.src = 53;
outport = *P*;
flags.loopback = 1;
output;

(This terminates ingress packet processing; the packet does not go to the next ingress table.)

Ingress table 31 External ports

Traffic from the **external** logical ports enter the ingress datapath pipeline via the **localnet** port. This table adds the below logical flows to handle the traffic from these ports.

- A priority−100 flow is added for each **external** logical port which doesn't reside on a chassis to drop the ARP/IPv6 NS request to the router IP(s) (of the logical switch) which matches on the **inport** of the **external** logical port and the valid **eth.src** address(es) of the **external** logical port.

  This flow guarantees that the ARP/NS request to the router IP address from the external ports is responded by only the chassis which has claimed these external ports. All the other chassis, drops these packets.

  A priority−100 flow is added for each **external** logical port which doesn't reside on a chassis to drop any packet destined to the router mac - with the match **inport ==** *external* **&& eth.src ==** *E* **&& eth.dst ==** *R* **&& !is_chassis_resident("***external***")** where *E* is the external port mac and *R* is the router port mac.

- A priority−0 flow that matches all packets to advances to table 20.

Ingress Table 32 Destination Lookup

This table implements switching behavior. It contains these logical flows:

- A priority−110 flow with the match **eth.src ==** *E* for all logical switch datapaths and applies the action **handle_svc_check(inport)**. Where *E* is the service monitor mac defined in the **options:svc_monitor_mac** column of **NB_Global** table.
- For each load balancer backend that is health checked from an address belonging to a logical router port, a priority−110 flow matching the probe reply on **inport == B** and applying **handle_svc_check(inport)**, where *B* is the backend's logical switch port.
- A **type=external** port has no VIF of its own, so its replies reach the switch through a localnet port and the flow above cannot match. For such a backend an additional copy of the flow is added for each localnet port *L* on the switch, matching **inport == L** instead and applying **inport = B; handle_svc_check(inport);** so that **ovn−controller** can tell which service monitor the reply belongs to. The action list ends there, so the assigned **inport** is not visible to any later table.
- A priority−100 flow that punts all IGMP/MLD packets to **ovn−controller** if multicast snooping is enabled on the logical switch.
- A priority−100 flow that forwards all DHCP broadcast packets coming from VIFs to the logical router port's MAC when DHCP relay is enabled on the logical switch.
- A priority−100 flow that matches **reg8[23] == 1** and does **output** action. This ensures that packets that got injected back into this table from egress table **Network Function** (after it set the **outport** for packet redirection) get forwarded without any further processing.

- For any logical port that's defined as a target of routing protocol redirecting (via **routing−protocol−redirect** option set on Logical Router Port), we redirect the traffic related to protocols specified in **routing−protocols** option. It's acoomplished with following priority−100 flows:
  - Flows that match Logical Router Port's IPs and destination port of the routing daemon are redirected to this port to allow external peers' connection to the daemon listening on this port.
  - Flows that match Logical Router Port's IPs and source port of the routing daemon are redirected to this port to allow replies from the peers.

  In addition to this, we add priority−100 rules that **clone** ARP replies and IPv6 Neighbor Advertisements to this port as well. These allow to build proper ARP/IPv6 neighbor list on this port.
- Priority−90 flows for each IPv4 address/VIP/NAT address owned by a router port connected to the switch. These flows match GARP packets for the specific IP addresses. Matched packets are forwarded to the **MC_FLOOD_L2** multicast group which contains all non-router logical ports.
- Priority−90 flows for transit switches that forward registered IP multicast traffic to their corresponding multicast group , which **ovn−northd** creates based on learnt **IGMP_Group** entries.
- Priority−90 flows that forward registered IP multicast traffic to their corresponding multicast group, which **ovn−northd** creates based on learnt **IGMP_Group** entries. The flows also forward packets to the **MC_MROUTER_FLOOD** multicast group, which **ovn−nortdh** populates with all the logical ports that are connected to logical routers with **options**:mcast_relay='true'.
- A priority−85 flow that forwards all IP multicast traffic destined to 224.0.0.X to the **MC_FLOOD_L2** multicast group, which **ovn−northd** populates with all non-router logical ports.
- A priority−85 flow that forwards all IP multicast traffic destined to reserved multicast IPv6 addresses (RFC 4291, 2.7.1, e.g., Solicited-Node multicast) to the **MC_FLOOD** multicast group, which **ovn−northd** populates with all enabled logical ports.
- A priority−80 flow that forwards all unregistered IP multicast traffic to the **MC_STATIC** multicast group, which **ovn−northd** populates with all the logical ports that have **options :mcast_flood='true'**. The flow also forwards unregistered IP multicast traffic to the **MC_MROUTER_FLOOD** multicast group, which **ovn−northd** populates with all the logical ports connected to logical routers that have **options :mcast_relay='true'**.
- A priority−80 flow that drops all unregistered IP multicast traffic if **other_config :mcast_snoop='true'** and **other_config :mcast_flood_unregistered='false'** and the switch is not connected to a logical router that has **options :mcast_relay='true'** and the switch doesn't have any logical port with **options :mcast_flood='true'**.
- Priority−80 flows for each IP address/VIP/NAT address owned by a router port connected to the switch. These flows match ARP requests and ND packets for the specific IP addresses. Matched packets are forwarded only to the router that owns the IP address and to the **MC_UNKNOWN** multicast group which contains all enabled logical ports that accept unknown destination packets.
- Priority−75 flows for each port connected to a logical router matching self originated ARP request/RARP request/ND packets. These packets are flooded to the **MC_FLOOD_L2** which contains all non-router logical ports.

- A priority−72 flow that outputs all ND NA (Neighbor Advertisement), ND RS (Router Solicitation) and ND RA (Router Advertisement) packets with an Ethernet broadcast or multicast **eth.dst** to the **MC_FLOOD** multicast group, which includes all ports. ND NA must reach routers for neighbor learning; ND RS must reach routers so they can respond with Router Advertisements; ND RA must reach routers for proper IPv6 network operation.
- A priority−72 flow that outputs all ARP requests and ND NS (Neighbor Solicitation) packets with an Ethernet broadcast or multicast **eth.dst** to the **MC_UNKNOWN** multicast group if **other_config:broadcast−arps−to−all−routers=false**.
- A priority−71 flow that outputs all ARP packets with an Ethernet broadcast or multicast **eth.dst** to the **MC_FLOOD** multicast group.
- A priority−71 flow that outputs all IP packets with an Ethernet broadcast or multicast **eth.dst** to the **MC_FLOOD_L2** multicast group, which contains only non-router logical ports. If any connected router has **options:mcast_relay=true**, the packet is also cloned to the **MC_MROUTER_FLOOD** multicast group (which contains only the router ports with relay enabled). If any port has **options:mcast_flood=true**, it is also cloned to the **MC_STATIC** multicast group. This prevents IP multicast from being unnecessarily forwarded to routers that would drop it.
- A priority−70 flow that outputs all packets with an Ethernet broadcast or multicast **eth.dst** to the **MC_FLOOD_L2** multicast group.
- One priority−50 flow that matches each known Ethernet address against **eth.dst**. Action of this flow outputs the packet to the single associated output port if it is enabled. **drop;** action is applied if LSP is disabled. If the logical switch port of type VIF has the option **options:pkt_clone_type** is set to the value **mc_unknown**, then the packet is also forwarded to the **MC_UNKNOWN** multicast group.

  The above flow is not added if the logical switch port is of type VIF, has **unknown** as one of its address and has the option **options:force_fdb_lookup** set to true.

  For the Ethernet address on a logical switch port of type **router**, when that logical switch port's **addresses** column is set to **router** and the connected logical router port has a gateway chassis:

  - The flow for the connected logical router port's Ethernet address is only programmed on the gateway chassis.
  - If the logical router has rules specified in **nat** with **external_mac**, then those addresses are also used to populate the switch's destination lookup on the chassis where **logical_port** is resident.

  For the Ethernet address on a logical switch port of type **router**, when that logical switch port's **addresses** column is set to **router** and the connected logical router port specifies a **reside−on−redirect−chassis** and the logical router to which the connected logical router port belongs to has a distributed gateway LRP:

  - The flow for the connected logical router port's Ethernet address is only programmed on the gateway chassis.

  For each forwarding group configured on the logical switch datapath, a priority−50 flow that matches on **eth.dst ==** *VIP*
  with an action of **fwd_group(childports=***args* **)**, where *args* contains comma separated logical switch child ports to load balance to. If **liveness** is enabled, then action also includes **liveness=true**.
- One priority−0 fallback flow that matches all packets with the action **outport = get_fdb(eth.dst); next;**. The action **get_fdb** gets the port for the **eth.dst** in the MAC learning table of the logical switch datapath. If there is no entry for **eth.dst** in the MAC

learning table, then it stores **none** in the **outport**.

Ingress Table 33 Destination unknown

This table handles the packets whose destination was not found or and looked up in the MAC learning table of the logical switch datapath. It contains the following flows.

- Priority 50 flow with the match **outport ==** *P* is added for each disabled Logical Switch Port **P**. This flow has action **drop;**.
- If the logical switch has logical ports with 'unknown' addresses set, then the below logical flow is added
  - Priority 50 flow with the match **outport == "none"** then outputs them to the **MC_UNKNOWN** multicast group, which **ovn−northd** populates with all enabled logical ports that accept unknown destination packets. As a small optimization, if no logical ports accept unknown destination packets, **ovn−northd** omits this multicast group and logical flow.

  If the logical switch has no logical ports with 'unknown' address set, then the below logical flow is added
  - Priority 50 flow with the match **outport == none** and drops the packets.
- One priority−0 fallback flow that outputs the packet to the egress stage with the outport learnt from **get_fdb** action.

Egress Table 0: Lookup MAC address learning table

This is similar to ingress table **Lookup MAC address learning table**
with the difference that MAC address learning lookup is only happening for ports with type **remote** whose port security is disabled and 'unknown' address set. This stage facilitates MAC learning on a transit switch connecting multiple availability zones.

Egress Table 1: Learn MAC of 'unknown' ports.

This is similar to ingress table **Learn MAC of 'unknown' ports**
with the difference that MAC address learning is only happening for ports with type **remote** whose port security is disabled and 'unknown' address set. This stage facilitates MAC learning on a transit switch connecting multiple availability zones.

Egress Table 2: **to−lport** *Pre-ACLs*

This is similar to ingress table **Pre−ACLs** except for **to−lport** traffic.

This table also has a priority−110 flow with the match **eth.src ==** *E* for all logical switch datapaths to move traffic to the next table. Where *E* is the service monitor mac defined in the **options:svc_monitor_mac** column of **NB_Global** table.

This table also has a priority−110 flow with the match **outport ==** *I* for all logical switch datapaths to move traffic to the next table. Where *I* is the peer of a logical router port. This flow is added to skip the connection tracking of packets which will be entering logical router datapath from logical switch datapath for routing.

This table also has a priority−110 flow for each network_function **inport** *P* that matches **inport ==** *P*. The action is to skip all the egress tables up to the **Network Function** table and advance the packet directly to the table after that. This is for the case where packet redirection happens in egress **Network Function** table. The same packet when it comes out of the other port of network function, they should not be processed again by the same egress stages, specially they should skip the conntrack processing.

Egress Table 3: Pre-LB

This table is similar to ingress table **Pre−LB**. It contains a priority−0 flow that simply moves traffic to the next table. Moreover it contains two priority−110 flows to move multicast, IPv6 Neighbor Discovery and MLD traffic to the next table. If any load balancing rules exist for the datapath, a priority−100 flow is added with a match of **ip** and action of **reg0[2] = 1; next;** to act as a hint for table **Pre−stateful** to send IP packets

to the connection tracker for packet de-fragmentation and possibly DNAT the destination VIP to one of the selected backend for already committed load balanced traffic.

This table also has a priority−110 flow with the match **eth.src ==** *E* for all logical switch datapaths to move traffic to the next table. Where *E* is the service monitor mac defined in the **options:svc_monitor_mac** column of **NB_Global** table.

This table also has a priority−110 flow with the match **outport ==** *I* for all logical switch datapaths to move traffic to the next table, and, if there are no stateful_acl, clear the ct_state. Where *I* is the peer of a logical router port. This flow is added to skip the connection tracking of packets which will be entering logical router datapath from logical switch datapath for routing.

When **enable−stateless−acl−with−lb** is enabled, additional priority−115 flow is added to match traffic with **REGBIT_ACL_STATELESS** set and pass connection tracking.

Egress Table 4: Pre-stateful

This is similar to ingress table **Pre−stateful**. This table adds the below 3 logical flows.

- A Priority−120 flow that send the packets to connection tracker using **ct_lb_mark;** as the action so that the already established traffic gets unDNATted from the backend IP to the load balancer VIP based on a hint provided by the previous tables with a match for **reg0[2] == 1**. If the packet was not DNATted earlier, then **ct_lb_mark** functions like **ct_next**.
- A priority−100 flow sends the packets to connection tracker based on a hint provided by the previous tables (with a match for **reg0[0] == 1**) by using the **ct_next;** action.
- A priority−0 flow that matches all packets to advance to the next table.

Egress Table 5: **from−lport** *ACL hints*

This is similar to ingress table **ACL hints**.

Egress Table 6: **to−lport** *ACL evaluation*

This is similar to ingress table **ACL eval** except for **to−lport** ACLs. As a reminder, these flows use the following register bits to indicate their verdicts. **Allow−type** ACLs set **reg8[16]**, **drop** ACLs set **reg8[17]**, and **reject** ACLs set **reg8[18]**.

Also like with ingress ACLs, egress ACLs can have network_function_group *id* and in that case the flow will set **reg8[21] = 1; reg8[22] = 1; reg0[22..29] =** *id*. These registers are used in the **Network Function** table.

Also like with ingress ACLs, egress ACLs can have a configured **tier**. If a tier is configured, then the current tier counter is evaluated against the ACL's configured tier in addition to the ACL's match. The current tier counter is stored in **reg8[30..31]**.

Similar to ingress table, a priority−65532 flow is added to allow IPv6 Neighbor solicitation, Neighbor discover, Router solicitation, Router advertisement and MLD packets regardless of other ACLs defined.

In addition, the following flows are added.

- A priority 34000 logical flow is added for each logical port which has DHCPv4 options defined to allow the DHCPv4 reply packet and which has DHCPv6 options defined to allow the DHCPv6 reply packet from the **Ingress Table 26: DHCP responses**. This is indicated by setting the allow bit.
- A priority 34000 logical flow is added for each logical switch datapath configured with DNS records with the match **udp.dst = 53** to allow the DNS reply packet from the **Ingress Table 28: DNS responses**. This is indicated by setting the allow bit.
- A priority 34000 logical flow is added for each logical switch datapath with the match **eth.src =** *E* to allow the service monitor request packet generated by **ovn−controller** with the action **next**, where *E* is the service monitor mac defined in the **options:svc_monitor_mac** column of **NB_Global** table. This is indicated by setting the allow bit.

Egress Table 7: **to−lport** *ACL sampling*

This is similar to ingress table **ACL sampling**.

Egress Table 8: **to−lport** *ACL action*

This is similar to ingress table **ACL action**.

Egress Table 9: Mirror

Overlay remote mirror table contains the following logical flows:

- For each logical switch port with an attached mirror, a logical flow with a priority of 100 is added. This flow matches all outcoming packets to the attached port, clones them, and forwards the cloned packets to the mirror target port.
- A priority 0 flow is added which matches on all packets and applies the action **next;**.
- A logical flow added for each Mirror Rule in Mirror table attached to logical switch ports, matches all outcoming packets that match rules and clones the packet and sends cloned packet to mirror target port.

Egress Table 10: **to−lport** *QoS*

This is similar to ingress table **QoS** except they apply to **to−lport** QoS rules.

Egress Table 11: Pre Network Function

This stage selects the active network function from a **Network_Function_Group** based on the network function group ID set by the ACL eval stage earlier. This stage is applicable for request packets of **to−lport** ACLs (**reg8[22] == 1**). Response packets for **from−lport** ACLs bypass this stage and use **ct_label.nf_id** directly in the Network Function table.

A network function group can contain one or more network functions. Health monitoring is performed by sending datapath probes as per parameters defined in **Network_Function_Health_Check**. This stage selects one of the healthy network functions. If none are healthy, the behavior follows the **fallback** column configured in the **Network_Function_Group** table. If health monitoring is not configured, any one from the group is selected.

When a request packet matches a **to−lport** ACL with **network_function_group** set, the ACL eval stage sets **reg8[21] = 1** (NF enabled), **reg8[22] = 1** (request direction), and **reg0[22..29]** to the network function group ID. This table then selects the active network function from the group and overwrites **reg0[22..29]** with the specific **id** of a **Network_Function** table entry. The subsequent Network Function table uses this NF ID to redirect packets to the appropriate network function port. In the future, this stage will be extended to support network function load balancing.

- For each network function group with *id* that has an active network function, a priority−99 flow matches **reg8[21] == 1 && reg8[22] == 1 && reg0[22..29] ==** *id* and sets **reg0[22..29] =** *nf_id***; next;** where *nf_id* is the **id** of the active **Network_Function** selected from the group. This prepares request packets that matched a **to−lport** ACL with network_function_group for redirection in the subsequent Network Function table.
- For each network function group with *id* that has **fallback** set to **fail−open**, a priority−10 flow matches **reg8[21] == 1 && reg8[22] == 1 && reg0[22..29] ==** *id* and sets **reg8[21] = 0; reg0[22..29] = 0; next;**. This clears both the NF enabled bit and the NF group ID, allowing packets to continue processing through the pipeline without network function redirection when no active network function is available (fail-open behavior).
- A priority−1 flow matches **reg8[21] == 1 && reg8[22] == 1** and sets **reg0[22..29] = 0; next;**. This is a catch-all flow for network function groups with **fallback** set to **fail−close** (or default) when no active network function is available. It clears only the NF group ID, leaving the NF enabled bit set. These packets will be dropped by the priority−1 drop rule in the subsequent Network Function table (fail-close behavior).

- A priority−0 flow that simply moves traffic to the next table.

Egress Table 12: Stateful

This is similar to ingress table **Stateful** except that there are no rules added for load balancing new connections. When **enable−stateless−acl−with−lb** is enabled, new stateless connections bypass connection tracking.

- A priority 120 flow is added for each network function port *P* that is identical to the priority 100 flow except for additional match **outport ==** *P* and additional action **ct_label.tun_if_id = reg5[16..31]**. In case packets redirected by network function logic gets tunneled from host1 to host2 where the network function port resides, host2's physical table 0 populates reg5[16..31] with the openflow tunnel interface id on which the packet was received. This priority 120 flow commits the tunnel id to the ct_label. That way, when the same packet comes out of the other port of the network function it can retrieve this information from the peer port's CT entry and tunnel the packet back to host1. This is required to make cross host traffic redirection work for VLAN subnet.

Egress Table 13: Network Function

This table handles request packets for **to−lport** ACLs and response packets for **from−lport** ACLs. For **to−lport** ACLs, request packets are redirected to the network function's **outport**, and corresponding response/related packets are handled in the ingress pipeline. For **from−lport** ACLs, request packets are handled in the ingress pipeline, but corresponding response/related packets for those flows are redirected here using the network function ID stored in **ct_label.nf_id** during request processing.

- Similar to ingress **Network Function** a priority−100 flow is added for each network_function port, that matches the inport with the network function port and advances the packet to the next table.
- For each active network function with *id* that is referenced in a network function group, a priority−99 flow matches **reg8[21] == 1 && reg8[22] == 1 && reg0[22..29] ==** *id* and sets **outport=***P***; reg8[23] = 1; next(pipeline=ingress, table=***T***)** where *P* is the **outport** of that network function and *T* is the ingress table **Destination Lookup**. This redirects request packets matching **to−lport** ACLs with network_function_group to the specific network function selected by the Pre Network Function stage. The packets are injected back to the ingress pipeline from where they get sent out, skipping any further lookup because of **reg8[23]**.
- For each active network function with *id* that is referenced in a network function group, a priority−99 rule matches **reg8[21] == 1 && reg8[22] == 0 && ct_label.nf_id ==** *id* and takes identical action as above. This redirects response and related packets for **from−lport** ACLs to the same network function that handled the request, using the NF ID stored in the connection tracking label.
- In each of the above cases, when the same packet comes out unchanged through the other port of the network_function, it would match the priority 100 flow and be forwarded to the next table.
- One priority−100 multicast match flow same as ingress **Network Function**.
- One priority−1 flow same as ingress **Network Function**.
- One priority−0 flow same as ingress **Network Function**.

Egress Table 14: Egress Port Security - check

This is similar to the port security logic in table **Ingress Port Security check** except that action **check_out_port_sec** is used to check the port security rules. This table adds the below logical flows.

- A priority 100 flow which matches on the multicast traffic and applies the action **REGBIT_PORT_SEC_DROP" = 0; next;"** to skip the out port security checks.

- A priority 0 logical flow is added which matches on all the packets and applies the action **REGBIT_PORT_SEC_DROP" = check_out_port_sec(); next;"**. The action **check_out_port_sec** applies the port security rules based on the addresses defined in the **port_security** column of **Logical_Switch_Port** table before delivering the packet to the **outport**.

Egress Table 15: Egress Port Security - Apply

This is similar to the ingress port security logic in ingress table **A Ingress Port Security – Apply**. This table drops the packets if the port security check failed in the previous stage i.e the register bit **REGBIT_PORT_SEC_DROP** is set to 1.

The following flows are added.

- For each port configured with egress qos in the **options:qdisc_queue_id** column of **Logical_Switch_Port**, running a localnet port on the same logical switch, a priority 110 flow is added which matches on the localnet **outport** and on the port **inport** and applies the action **set_queue(id); output;"**.
- For each localnet port configured with egress qos in the **options:qdisc_queue_id** column of **Logical_Switch_Port**, a priority 100 flow is added which matches on the localnet **outport** and applies the action **set_queue(id); output;"**.

  Please remember to mark the corresponding physical interface with **ovn−egress−iface** set to true in **external_ids**.
- A priority−50 flow that drops the packet if the register bit **REGBIT_PORT_SEC_DROP** is set to 1.
- A priority−0 flow that outputs the packet to the **outport**.

Logical Router Datapaths

Logical router datapaths will only exist for **Logical_Router** rows in the **OVN_Northbound** database that do not have **enabled** set to **false**

Ingress Table 0: L2 Admission Control

This table drops packets that the router shouldn't see at all based on their Ethernet headers. It contains the following flows:

- Priority−100 flows to drop packets with VLAN tags or multicast Ethernet source addresses.
- For each enabled router port *P* with Ethernet address *E*, a priority−50 flow that matches **inport == *P* && (eth.mcast || eth.dst == *E***), stores the router port ethernet address and advances to next table, with action **xreg0[0..47]=E; next;**.

  For the gateway port on a distributed logical router (where one of the logical router ports specifies a gateway chassis), the above flow matching **eth.dst == *E*** is only programmed on the gateway port instance on the gateway chassis. If LRP's logical switch has attached LSP of **vtep** type, the **is_chassis_resident()** part is not added to lflow to allow traffic originated from logical switch to reach LR services (LBs, NAT).

  For each gateway port *GW* on a distributed logical router a priority−120 flow that matches 'recirculated' icmp{4,6} error 'packet too big' and **eth.dst == *D* && !is_chassis_resident(** *cr-GW***)** where *D* is the gateway port mac address and *cr-GW* is the chassis resident port of *GW*, swap inport and outport and stores *GW* as inport.

  This table adds a priority−105 flow that matches 'recirculated' icmp{4,6} error 'packet too big' to drop the packet.

  For unicast L2 traffic on a distributed logical router or for gateway router where the port is configured with **options:gateway_mtu** the action of the above flow is modified adding **check_pkt_larger** in order to mark the packet setting **REGBIT_PKT_LARGER** if the size is greater than the MTU. If the port is also configured with

options:gateway_mtu_bypass then another flow is added, with priority−55, to bypass the **check_pkt_larger** flow. This is useful for traffic that normally doesn't need to be fragmented and for which check_pkt_larger, which might not be offloadable, is not really needed. One such example is TCP traffic.

- For each **dnat_and_snat** NAT rule on a distributed router that specifies an external Ethernet address *E*, a priority−50 flow that matches **inport ==** *GW* **&& eth.dst ==** *E*, where *GW* is the logical router distributed gateway port corresponding to the NAT rule (specified or inferred), with action **xreg0[0..47]=E; next;**.

  This flow is only programmed on the gateway port instance on the chassis where the **logical_port** specified in the NAT rule resides.

- A priority−0 logical flow that matches all packets not already handled (match **1**) and drops them (action **drop;**).

Other packets are implicitly dropped.

Ingress Table 1: Neighbor lookup

For ARP and IPv6 Neighbor Discovery packets, this table looks into the **MAC_Binding** records to determine if OVN needs to learn the mac bindings. Following flows are added:

- For each router port *P* that owns IP address *A*, which belongs to subnet *S* with prefix length *L*, if the option **always_learn_from_arp_request** is **true** for this router, a priority−100 flow is added which matches **inport ==** *P* **&& arp.spa ==** *S/L* **&& arp.op == 1** (ARP request) with the following actions:

  reg9[2] = lookup_arp(inport, arp.spa, arp.sha);
  next;

  If the option **always_learn_from_arp_request** is **false**, the following two flows are added.

  A priority−110 flow is added which matches **inport ==** *P* **&& arp.spa ==** *S/L* **&& arp.tpa ==** *A* **&& arp.op == 1** (ARP request) with the following actions:

  reg9[2] = lookup_arp(inport, arp.spa, arp.sha);
  reg9[3] = 1;
  next;

  A priority−100 flow is added which matches **inport ==** *P* **&& arp.spa ==** *S/L* **&& arp.op == 1** (ARP request) with the following actions:

  reg9[2] = lookup_arp(inport, arp.spa, arp.sha);
  reg9[3] = lookup_arp_ip(inport, arp.spa);
  next;

  If the logical router port *P* is a distributed gateway router port, additional match **is_chassis_resident(cr−***P***)** is added for all these flows.

- A priority−100 flow which matches on ARP reply packets and applies the actions if the option **always_learn_from_arp_request** is **true**:

  reg9[2] = lookup_arp(inport, arp.spa, arp.sha);
  next;

  If the option **always_learn_from_arp_request** is **false**, the above actions will be:

  reg9[2] = lookup_arp(inport, arp.spa, arp.sha);
  reg9[3] = 1;

next;

- A priority−100 flow which matches on IPv6 Neighbor Discovery advertisement packet and applies the actions if the option **always_learn_from_arp_request** is **true**:

  reg9[2] = lookup_nd(inport, nd.target, nd.tll);
  next;

  If the option **always_learn_from_arp_request** is **false**, the above actions will be:

  reg9[2] = lookup_nd(inport, nd.target, nd.tll);
  reg9[3] = 1;
  next;

- A higher priority−105 flow which matches on IPv6 Neighbor Discovery advertisement packet with **nd.tll == 0** and applies the actions if the option **>always_learn_from_arp_request** is **>true**:

  reg9[2] = lookup_nd(inport, nd.target, eth.src);
  next;

  If the option **always_learn_from_arp_request>** is **false**, a priority−115 flow is added instead:

  reg9[2] = lookup_nd(inport, nd.target, eth.src);
  reg9[3] = lookup_nd_ip(inport, nd.target);
  next;

- A priority−100 flow which matches on IPv6 Neighbor Discovery solicitation packet and applies the actions if the option **always_learn_from_arp_request** is **true**:

  reg9[2] = lookup_nd(inport, ip6.src, nd.sll);
  next;

  If the option **always_learn_from_arp_request** is **false**, the above actions will be:

  reg9[2] = lookup_nd(inport, ip6.src, nd.sll);
  reg9[3] = lookup_nd_ip(inport, ip6.src);
  next;

- A priority−0 fallback flow that matches all packets and applies the action **reg9[2] = 1; next;** advancing the packet to the next table.

Ingress Table 2: Neighbor learning

This table adds flows to learn the mac bindings from the ARP and IPv6 Neighbor Solicitation/Advertisement packets if it is needed according to the lookup results from the previous stage.

reg9[2] will be **1** if the **lookup_arp/lookup_nd** in the previous table was successful or skipped, meaning no need to learn mac binding from the packet.

reg9[3] will be **1** if the **lookup_arp_ip/lookup_nd_ip** in the previous table was successful or skipped, meaning it is ok to learn mac binding from the packet (if reg9[2] is 0).

- A priority−100 flow with the match **reg9[2] == 1 || reg9[3] == 0** and advances the packet to the next table as there is no need to learn the neighbor.

- A priority−95 flow with the match **nd_ns && (ip6.src == 0 || nd.sll == 0)** and applies the action **next;**
- A priority−90 flow with the match **arp** and applies the action **put_arp(inport, arp.spa, arp.sha); next;**
- A priority−95 flow with the match **nd_na && nd.tll == 0** and applies the action **put_nd(inport, nd.target, eth.src); next;**
- A priority−90 flow with the match **nd_na** and applies the action **put_nd(inport, nd.target, nd.tll); next;**
- A priority−90 flow with the match **nd_ns** and applies the action **put_nd(inport, ip6.src, nd.sll); next;**
- A priority−0 logical flow that matches all packets not already handled (match **1**) and drops them (action **drop;**).

Ingress Table 3: IP Input

This table is the core of the logical router datapath functionality. It contains the following flows to implement very basic IP host functionality.

- For each **dnat_and_snat** NAT rule on a distributed logical routers or gateway routers with gateway port configured with **options:gateway_mtu** to a valid integer value *M*, a priority−160 flow with the match **inport ==** *LRP* **&& REGBIT_PKT_LARGER && REGBIT_EGRESS_LOOPBACK == 0**, where *LRP* is the logical router port and applies the following action for ipv4 and ipv6 respectively:

  icmp4_error {
  icmp4.type = 3; /* Destination Unreachable. */
  icmp4.code = 4; /* Frag Needed and DF was Set. */
  icmp4.frag_mtu = *M*;
  eth.dst = eth.src;
  eth.src = *E*;
  ip4.dst = ip4.src;
  ip4.src = *I*;
  ip.ttl = 255;
  REGBIT_EGRESS_LOOPBACK = 1;
  REGBIT_PKT_LARGER 0;
  outport = *LRP*;
  flags.loopback = 1;
  output;
  };
  icmp6_error {
  icmp6.type = 2;
  icmp6.code = 0;
  icmp6.frag_mtu = *M*;
  eth.dst = eth.src;
  eth.src = *E*;
  ip6.dst = ip6.src;
  ip6.src = *I*;
  ip.ttl = 255;
  REGBIT_EGRESS_LOOPBACK = 1;
  REGBIT_PKT_LARGER 0;
  outport = *LRP*;
  flags.loopback = 1;
  output;
  };

where *E* and *I* are the NAT rule external mac and IP respectively.

- For distributed logical routers or gateway routers with gateway port configured with **options:gateway_mtu** to a valid integer value, a priority−150 flow with the match **inport == *LRP* && REGBIT_PKT_LARGER && REGBIT_EGRESS_LOOPBACK == 0**, where *LRP* is the logical router port and applies the following action for ipv4 and ipv6 respectively:

  icmp4_error {
  icmp4.type = 3; /* Destination Unreachable. */
  icmp4.code = 4; /* Frag Needed and DF was Set. */
  icmp4.frag_mtu = *M*;
  eth.dst = *E*;
  ip4.dst = ip4.src;
  ip4.src = *I*;
  ip.ttl = 255;
  REGBIT_EGRESS_LOOPBACK = 1;
  REGBIT_PKT_LARGER 0;
  next(pipeline=ingress, table=0);
  };
  icmp6_error {
  icmp6.type = 2;
  icmp6.code = 0;
  icmp6.frag_mtu = *M*;
  eth.dst = *E*;
  ip6.dst = ip6.src;
  ip6.src = *I*;
  ip.ttl = 255;
  REGBIT_EGRESS_LOOPBACK = 1;
  REGBIT_PKT_LARGER 0;
  next(pipeline=ingress, table=0);
  };

- For each NAT entry of a distributed logical router (with distributed gateway router port(s)) of type **snat**, a priority−120 flow with the match **inport ==** *P* **&& ip4.src ==** *A* advances the packet to the next pipeline, where *P* is the distributed logical router port corresponding to the NAT entry (specified or inferred) and *A* is the **external_ip** set in the NAT entry. If *A* is an IPv6 address, then **ip6.src** is used for the match.

  The above flow is required to handle the routing of the East/west NAT traffic.

- For each BFD port the two following priority−110 flows are added to manage BFD traffic:

  - if **ip4.src** or **ip6.src** is any IP address owned by the router port and **udp.dst == 3784** , the packet is advanced to the next pipeline stage.
  - if **ip4.dst** or **ip6.dst** is any IP address owned by the router port and **udp.dst == 3784** , the **handle_bfd_msg** action is executed.

- For each logical router port configured with DHCP relay the following priority−110 flows are added to manage the DHCP relay traffic:

  - if **inport** is lrp and **ip4.src == 0.0.0.0** and **ip4.dst == 255.255.255.255** and **ip4.frag == 0** and **udp.src == 68** and **udp.dst == 67**, the **dhcp_relay_req_chk** action is executed.

reg9[7] = dhcp_relay_req_chk(*lrp_ip*,
dhcp_server_ip);next

if action is successful then, GIADDR in the dhcp header is updated with lrp ip and stores 1 into reg9[7] else stores 0 into reg9[7].

- if **ip4.src** is DHCP server ip and **ip4.dst** is lrp IP and **udp.src == 67** and **udp.dst == 67**, the packet is advanced to the next pipeline stage.

- L3 admission control: Priority−120 flows allows IGMP and MLD packets if the router has logical ports that have **options :mcast_flood='true'**.
- L3 admission control: A priority−100 flow drops packets that match any of the following:
  - **ip4.src[28..31] == 0xe** (multicast source)
  - **ip4.src == 255.255.255.255** (broadcast source)
  - **ip4.src == 127.0.0.0/8 || ip4.dst == 127.0.0.0/8** (localhost source or destination)
  - **ip4.src == 0.0.0.0/8 || ip4.dst == 0.0.0.0/8** (zero network source or destination)
  - **ip4.src** or **ip6.src** is any IP address owned by the router, unless the packet was recirculated due to egress loopback as indicated by **REGBIT_EGRESS_LOOPBACK**.
  - **ip4.src** is the broadcast address of any IP network known to the router. Point-to-point (**/31**, RFC 3021) networks have no broadcast address and are excluded, so that traffic from a **/31** peer is not dropped.
- A priority−100 flow parses DHCPv6 replies from IPv6 prefix delegation routers (**udp.src == 547 && udp.dst == 546**). The **handle_dhcpv6_reply** is used to send IPv6 prefix delegation messages to the delegation router.
- For each load balancer applied to this logical router configured with **VIP** template, a priority−100 flow matching **ip4.dst** or **ip6.dst** with the configured load balancer **VIP** and action **next;**. These flows avoid dropping the packet if the **VIP** is set to one of the router IPs.
- ICMP echo reply. These flows reply to ICMP echo requests received for the router's IP address. Let *A* be an IP address owned by a router port. Then, for each *A* that is an IPv4 address, a priority−90 flow matches on **ip4.dst ==** *A* and **icmp4.type == 8 && icmp4.code == 0** (ICMP echo request). For each *A* that is an IPv6 address, a priority−90 flow matches on **ip6.dst ==** *A* and **icmp6.type == 128 && icmp6.code == 0** (ICMPv6 echo request). The port of the router that receives the echo request does not matter. Also, the **ip.ttl** of the echo request packet is not checked, so it complies with RFC 1812, section 4.2.2.9. Flows for ICMPv4 echo requests use the following actions:

  ip4.dst <−> ip4.src;
  ip.ttl = 255;
  icmp4.type = 0;
  flags.loopback = 1;
  next;

  Flows for ICMPv6 echo requests use the following actions:

  ip6.dst <−> ip6.src;
  ip.ttl = 255;
  icmp6.type = 129;
  flags.loopback = 1;
  next;

- Reply to ARP requests.

  These flows reply to ARP requests for the router's own IP address. The ARP requests are handled only if the requestor's IP belongs to the same subnets of the logical router port. For each router port *P* that owns IP address *A*, which belongs to subnet *S* with prefix length *L*, and Ethernet address *E*, a priority−90 flow matches **inport == *P* && arp.spa == *S*/*L* && arp.op == 1 && arp.tpa == *A*** (ARP request) with the following actions:

  eth.dst = eth.src;
  eth.src = xreg0[0..47];
  arp.op = 2; /* ARP reply. */
  arp.tha = arp.sha;
  arp.sha = xreg0[0..47];
  arp.tpa = arp.spa;
  arp.spa = *A*;
  outport = inport;
  flags.loopback = 1;
  output;

  For the gateway port on a distributed logical router (where one of the logical router ports specifies a gateway chassis), the above flows are only programmed on the gateway port instance on the gateway chassis. This behavior avoids generation of multiple ARP responses from different chassis, and allows upstream MAC learning to point to the gateway chassis.

  For the logical router port with the option **reside−on−redirect−chassis** set (which is centralized), the above flows are only programmed on the gateway port instance on the gateway chassis (if the logical router has a distributed gateway port). This behavior avoids generation of multiple ARP responses from different chassis, and allows upstream MAC learning to point to the gateway chassis.

- Reply to IPv6 Neighbor Solicitations. These flows reply to Neighbor Solicitation requests for the router's own IPv6 address and populate the logical router's mac binding table.

  For each router port *P* that owns IPv6 address *A*, solicited node address *S*, and Ethernet address *E*, a priority−90 flow matches **inport == *P* && nd_ns && ip6.dst == {*A*, *E*} && nd.target == *A*** with the following actions:

  nd_na_router {
  eth.src = xreg0[0..47];
  ip6.src = *A*;
  nd.target = *A*;
  nd.tll = xreg0[0..47];
  outport = inport;
  flags.loopback = 1;
  output;
  };

  For the gateway port on a distributed logical router (where one of the logical router ports specifies a gateway chassis), the above flows replying to IPv6 Neighbor Solicitations are only programmed on the gateway port instance on the gateway chassis. This behavior avoids generation of multiple replies from different chassis, and allows upstream MAC learning to point to the gateway chassis.

- These flows reply to ARP requests or IPv6 neighbor solicitation for the virtual IP addresses configured in the router for NAT (both DNAT and SNAT) or load balancing.

IPv4: For a configured NAT (both DNAT and SNAT) IP address or a load balancer IPv4 VIP *A*, for each router port *P* with Ethernet address *E*, a priority−90 flow matches **arp.op == 1 && arp.tpa ==** *A* (ARP request) with the following actions:

eth.dst = eth.src;
eth.src = xreg0[0..47];
arp.op = 2; /* ARP reply. */
arp.tha = arp.sha;
arp.sha = xreg0[0..47];
arp.tpa <−> arp.spa;
outport = inport;
flags.loopback = 1;
output;

IPv4: For a configured load balancer IPv4 VIP, a similar flow is added with the additional match **inport ==** *P* if the VIP is reachable from any logical router port of the logical router.

If the router port *P* is a distributed gateway router port, then the **is_chassis_resident(***P***)** is also added in the match condition for the load balancer IPv4 VIP *A*.

IPv6: For a configured NAT (both DNAT and SNAT) IP address or a load balancer IPv6 VIP *A* (if the VIP is reachable from any logical router port of the logical router), solicited node address *S*, for each router port *P* with Ethernet address *E*, a priority−90 flow matches **inport ==** *P* **&& nd_ns && ip6.dst == {***A***,** *S***} && nd.target ==** *A* with the following actions:

eth.dst = eth.src;
nd_na {
eth.src = xreg0[0..47];
nd.tll = xreg0[0..47];
ip6.src = *A*;
nd.target = *A*;
outport = inport;
flags.loopback = 1;
output;
}

If the router port *P* is a distributed gateway router port, then the **is_chassis_resident(***P***)** is also added in the match condition for the load balancer IPv6 VIP *A*.

For the gateway port on a distributed logical router with NAT (where one of the logical router ports specifies a gateway chassis):

- If the corresponding NAT rule cannot be handled in a distributed manner, then a priority−92 flow is programmed on the gateway port instance on the gateway chassis. A priority−91 drop flow is programmed on the other chassis when ARP requests/NS packets are received on the gateway port. This behavior avoids generation of multiple ARP responses from different chassis, and allows upstream MAC learning to point to the gateway chassis.
- If the corresponding NAT rule can be handled in a distributed manner, then this flow is only programmed on the gateway port instance where the **logical_port** specified in the NAT rule resides.

  Some of the actions are different for this case, using the **external_mac** specified in the NAT rule rather than the gateway port's Ethernet address *E*:

eth.src = *external_mac*;
arp.sha = *external_mac*;

or in the case of IPv6 neighbor solicition:

eth.src = *external_mac*;
nd.tll = *external_mac*;

This behavior avoids generation of multiple ARP responses from different chassis, and allows upstream MAC learning to point to the correct chassis.

- Priority−85 flows which drops the ARP and IPv6 Neighbor Discovery packets.
- A priority−84 flow explicitly allows IPv6 multicast traffic that is supposed to reach the router pipeline (i.e., router solicitation and router advertisement packets).
- A priority−83 flow explicitly drops IPv6 multicast traffic that is destined to reserved multicast groups.
- A priority−82 flow allows IP multicast traffic if **options**:mcast_relay='true', otherwise drops it.
- UDP port unreachable. Priority−80 flows generate ICMP port unreachable messages in reply to UDP datagrams directed to the router's IP address, except in the special case of gateways, which accept traffic directed to a router IP for load balancing and NAT purposes.

  These flows should not match IP fragments with nonzero offset.
- TCP reset. Priority−80 flows generate TCP reset messages in reply to TCP datagrams directed to the router's IP address, except in the special case of gateways, which accept traffic directed to a router IP for load balancing and NAT purposes.

  These flows should not match IP fragments with nonzero offset.
- Protocol or address unreachable. Priority−70 flows generate ICMP protocol or address unreachable messages for IPv4 and IPv6 respectively in reply to packets directed to the router's IP address on IP protocols other than UDP, TCP, and ICMP, except in the special case of gateways, which accept traffic directed to a router IP for load balancing purposes.

  These flows should not match IP fragments with nonzero offset.
- Drop other IP traffic to this router. These flows drop any other traffic destined to an IP address of this router that is not already handled by one of the flows above, which amounts to ICMP (other than echo requests) and fragments with nonzero offsets. For each IP address *A* owned by the router, a priority−60 flow matches **ip4.dst ==** *A* or **ip6.dst ==** *A* and drops the traffic. An exception is made and the above flow is not added if the router port's own IP address is used to SNAT packets passing through that router or if it is used as a load balancer VIP.

The flows above handle all of the traffic that might be directed to the router itself. The following flows (with lower priorities) handle the remaining traffic, potentially for forwarding:

- Drop Ethernet local broadcast. A priority−50 flow with match **eth.bcast** drops traffic destined to the local Ethernet broadcast address. By definition this traffic should not be forwarded.
- Avoid ICMP time exceeded for multicast. A priority−32 flow with match **ip.ttl == {0, 1} && !ip.later_frag && (ip4.mcast || ip6.mcast)** and actions **drop;** drops multicast packets whose TTL has expired without sending ICMP time exceeded.

- ICMP time exceeded. For each router port *P*, whose IP address is *A*, a priority−31 flow with match **inport == *P* && ip.ttl == {0, 1} && !ip.later_frag** matches packets whose TTL has expired, with the following actions to send an ICMP time exceeded reply for IPv4 and IPv6 respectively. If the router port **P** is a distributed gateway router port, then the **is_chassis_resident(cr−P)** is also added in the match condition:

  icmp4 {
  ** icmp4.type = 11; /* Time exceeded. */**
  ** icmp4.code = 0; /* TTL exceeded in transit. */**
  ** ip4.dst = ip4.src;**
  ** ip4.src =** *A*;
  ** ip.ttl = 254;**
  ** next;**
  };
  icmp6 {
  ** icmp6.type = 3; /* Time exceeded. */**
  ** icmp6.code = 0; /* TTL exceeded in transit. */**
  ** ip6.dst = ip6.src;**
  ** ip6.src =** *A*;
  ** ip.ttl = 254;**
  ** next;**
  };

- TTL discard. A priority−30 flow with match **ip.ttl == {0, 1}** and actions **drop;** drops other packets whose TTL has expired, that should not receive a ICMP error reply (i.e. fragments with nonzero offset).
- Next table. A priority−0 flows match all packets that aren't already handled and uses actions **next;** to feed them to the next table.

Ingress Table 4: DHCP Relay Request

This stage process the DHCP request packets on which **dhcp_relay_req_chk** action is applied in the IP input stage.

- A priority−100 logical flow is added for each logical router port configured with DHCP relay that matches **inport** is lrp and **ip4.src == 0.0.0.0** and **ip4.dst == 255.255.255.255** and **udp.src == 68**
  and **udp.dst == 67** and **reg9[7] == 1** and applies following actions. If **reg9[7]** is set to 1 then, **dhcp_relay_req_chk** action was successful.

  ip4.src=*lrp ip*;
  ip4.dst=*dhcp server ip*;
  udp.src = 67;
  next;

- A priority−1 logical flow is added for each logical router port configured with DHCP relay that matches **inport** is lrp and **ip4.src == 0.0.0.0** and **ip4.dst == 255.255.255.255** and **udp.src == 68**
  and **udp.dst == 67** and **reg9[7] == 0** and drops the packet. If **reg9[7]** is set to 0 then, **dhcp_relay_req_chk** action was unsuccessful.
- A priority−0 flow that matches all packets to advance to the next table.

Ingress Table 5: UNSNAT

This is for already established connections' reverse traffic. i.e., SNAT has already been done in egress pipeline and now the packet has entered the ingress pipeline as part of a reply. It is unSNATted here.

Ingress Table 5: UNSNAT on Gateway and Distributed Routers

- If the Router (Gateway or Distributed) is configured with load balancers, then below lflows are added:

  For each IPv4 address *A* defined as load balancer VIP with the protocol *P* (and the protocol port *T* if defined) is also present as an **external_ip** in the NAT table, a priority−120 logical flow is added with the match **ip4 && ip4.dst ==** *A* **&&** *P* with the action **next;** to advance the packet to the next table. If the load balancer has protocol port **B** defined, then the match also has *P***.dst ==** *B*.

  The above flows are also added for IPv6 load balancers.

Ingress Table 5: UNSNAT on Gateway Routers

- If the Gateway router has been configured to force SNAT any previously DNATted packets to *B*, a priority−110 flow matches **ip && ip4.dst ==** *B* or **ip && ip6.dst ==** *B* with an action **ct_snat;** .

  If the Gateway router is configured with **lb_force_snat_ip=router_ip** then for every logical router port *P* attached to the Gateway router with the router ip *B*, a priority−110 flow is added with the match **inport ==** *P* **&& ip4.dst ==** *B* or **inport ==** *P* **&& ip6.dst ==** *B* with an action **ct_snat;** .

  If the Gateway router has been configured to force SNAT any previously load-balanced packets to *B*, a priority−100 flow matches **ip && ip4.dst ==** *B* or **ip && ip6.dst ==** *B* with an action **ct_snat;** .

  For each NAT configuration in the OVN Northbound database, that asks to change the source IP address of a packet from *A* to *B*, a priority−90 flow matches **ip && ip4.dst ==** *B* or **ip && ip6.dst ==** *B* with an action **ct_snat;** . If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **next;**.

  A priority−0 logical flow with match **1** has actions **next;**.

Ingress Table 5: UNSNAT on Distributed Routers

- For each configuration in the OVN Northbound database, that asks to change the source IP address of a packet from *A* to *B*, two priority−100 flows are added.

  If the NAT rule cannot be handled in a distributed manner, then the below priority−100 flows are only programmed on the gateway chassis.

  - The first flow matches **ip && ip4.dst ==** *B* **&& inport ==** *GW*
    or **ip && ip6.dst ==** *B* **&& inport ==** *GW* where *GW* is the distributed gateway port corresponding to the NAT rule (specified or inferred), with an action **ct_snat;** to unSNAT in the common zone. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **next;**.

    If the NAT entry is of type **snat**, then there is an additional match **is_chassis_resident(***cr-GW***)**
    where *cr-GW* is the chassis resident port of *GW*.

  A priority−0 logical flow with match **1** has actions **next;**.

Ingress Table 6: POST USNAT

This is to check whether the packet is already tracked in SNAT zone. It contains a priority−0 flow that simply moves traffic to the next table.

If the **options:ct−commit−all** is set to **true** the following two flows are configured matching on **ip && ct.new** with an action **flags.unsnat_new = 1; next;** and **ip && !ct.trk** with an action **flags.unsnat_not_tracked = 1; next;** Which sets one of the flags that is used in later stages. There is extra match on both when there is configured DGP **inport == DGP && is_chassis_resident(CHASSIS)**.

Ingress Table 7: DEFRAG

This is to send packets to connection tracker for tracking and defragmentation. It contains a priority−0 flow that simply moves traffic to the next table.

For all load balancing rules that are configured in **OVN_Northbound** database for a Gateway router, a priority−100 flow is added for each configured virtual IP address *VIP*. For IPv4 *VIPs* the flow matches **ip && ip4.dst == *VIP***. For IPv6 *VIPs*, the flow matches **ip && ip6.dst == *VIP***. The flow applies the action **ct_dnat;** to send IP packets to the connection tracker for packet de-fragmentation and to dnat the destination IP for the committed connection before sending it to the next table.

If ECMP routes with symmetric reply are configured in the **OVN_Northbound** database for a gateway router, a priority−100 flow is added for each router port on which symmetric replies are configured. The matching logic for these ports essentially reverses the configured logic of the ECMP route. So for instance, a route with a destination routing policy will instead match if the source IP address matches the static route's prefix. The flow uses the actions **chk_ecmp_nh_mac(); ct_next** or **chk_ecmp_nh(); ct_next** to send IP packets to table **76** or to table **77** in order to check if source info are already stored by OVN and then to the connection tracker for packet de-fragmentation and tracking before sending it to the next table.

If load balancing rules are configured in **OVN_Northbound** database for a Gateway router, a priority 50 flow that matches **icmp || icmp6** with an action of **ct_dnat;**, this allows potentially related ICMP traffic to pass through CT.

If the **options:ct−commit−all** is set to **true** the following flow is configured matching on **ip && (!ct.trk || !ct.rpl)** with an action **ct_next(dnat);**. There is extra match when the LR is configured as DGP **inport == DGP && is_chassis_resident(CHASSIS)**.

Ingress Table 8: Connection tracking field extraction

This table extracts connection tracking fields for new connections and stores them in registers for use by subsequent load balancing stages.

- For all new connections (**ct.new**), a priority−100 flow extracts the connection tracking protocol and destination port information into registers:

  reg1[16..23] = ct_proto();
  reg1[0..15] = ct_tp_dst();
  next;

  This stores the connection tracking destination port in **REG_CT_TP_DST** (reg1[0..15]) and the protocol in **REG_CT_PROTO** (reg1[16..23]).

- A priority−0 flow that matches all packets and advances to the next table with action **next;**.

Ingress Table 9: Load balancing affinity check

Load balancing affinity check table contains the following logical flows:

- For all the configured load balancing rules for a logical router where a positive affinity timeout is specified in **options** column, that includes a L4 port *PORT* of protocol *P* and IPv4 or IPv6 address *VIP*, a priority−100 flow that matches on **ct.new && ip && ip.dst == *VIP* && REG_CT_PROTO == *P_NUM* && REG_CT_TP_DST == *PORT*** (**xxreg0 == *VIP*** in the IPv6 case) with an action of **reg0 = ip.dst; reg9[16..31] = P.dst; reg9[6] = chk_lb_aff(); next;** (**xxreg0 ==** *ip6.dst* in the IPv6 case), where *P_NUM* is the protocol number (6 for TCP, 17 for UDP, 132 for SCTP).

- A priority 0 flow is added which matches on all packets and applies the action **next;**.

Ingress Table 10: DNAT

Packets enter the pipeline with destination IP address that needs to be DNATted from a virtual IP address to a real IP address. Packets in the reverse direction needs to be unDNATed.

Ingress Table 8: Load balancing DNAT rules

Following load balancing DNAT flows are added for Gateway router or Router with gateway port. These flows are programmed only on the gateway chassis. These flows do not get programmed for load balancers with IPv6 *VIPs*.

- For all the configured load balancing rules for a logical router where a positive affinity timeout is specified in **options** column, that includes a L4 port *PORT* of protocol *P* and IPv4 or IPv6 address *VIP*, a priority−150 flow that matches on **reg9[6] == 1 && ct.new && ip && ip.dst ==** *VIP* **&& REG_CT_PROTO ==** *P_NUM* **&& REG_CT_TP_DST ==** *PORT* with an action of **ct_lb_mark(***args***)** , where *args* contains comma separated IP addresses (and optional port numbers) to load balance to, and *P_NUM* is the protocol number (6 for TCP, 17 for UDP, 132 for SCTP). The address family of the IP addresses of *args* is the same as the address family of *VIP*.

- If controller_event has been enabled for all the configured load balancing rules for a Gateway router or Router with gateway port in **OVN_Northbound** database that does not have configured backends, a priority−130 flow is added to trigger ovn-controller events whenever the chassis receives a packet for that particular VIP. If **event−elb** meter has been previously created, it will be associated to the empty_lb logical flow

- For all the configured load balancing rules for a Gateway router or Router with gateway port in **OVN_Northbound** database that includes a L4 port *PORT* of protocol *P* and IPv4 or IPv6 address *VIP*, a priority−120 flow that matches on **ct.new && !ct.rel && ip && ip.dst ==** *VIP* **&& REG_CT_PROTO ==** *P_NUM* **&& REG_CT_TP_DST ==** *PORT* with an action of **ct_lb_mark(***args***)**, where *args* contains comma separated IPv4 or IPv6 addresses (and optional port numbers) to load balance to, and *P_NUM* is the protocol number (6 for TCP, 17 for UDP, 132 for SCTP). If the router is configured to force SNAT any load-balanced packets, the above action will be replaced by **flags.force_snat_for_lb = 1; ct_lb_mark(***args***; force_snat);**. If the load balancing rule is configured with **skip_snat** set to true, the above action will be replaced by **flags.skip_snat_for_lb = 1; ct_lb_mark(***args***; skip_snat);**. If health check is enabled, then *args* will only contain those endpoints whose service monitor status entry in **OVN_Southbound** db is either **online** or empty.

- For all the configured load balancing rules for a router in **OVN_Northbound** database that includes just an IP address *VIP* to match on, a priority−110 flow that matches on **ct.new && !ct.rel && ip4 && ip.dst ==** *VIP* with an action of **ct_lb_mark(***args***)**, where *args* contains comma separated IPv4 or IPv6 addresses. If the router is configured to force SNAT any load-balanced packets, the above action will be replaced by **flags.force_snat_for_lb = 1; ct_lb_mark(***args***; force_snat);**. If the load balancing rule is configured with **skip_snat** set to true, the above action will be replaced by **flags.skip_snat_for_lb = 1; ct_lb_mark(***args***; skip_snat);**.

  The previous table **lr_in_defrag** sets the register **reg0** (or **xxreg0** for IPv6) and does **ct_dnat**. Hence for established traffic, this table just advances the packet to the next stage.

- If the load balancer is created with **−−reject** option and it has no active backends, a TCP reset segment (for tcp) or an ICMP port unreachable packet (for all other kind of traffic) will be sent whenever an incoming packet is received for this load-balancer. Please note using **−−reject** option will disable empty_lb SB controller event for this load balancer.

- For the related traffic, a priority 50 flow that matches **ct.rel && !ct.est && !ct.new** with an action of **ct_commit_nat;**, if the router has load balancer assigned to it. Along with two priority 70 flows that match **skip_snat** and **force_snat** flags, setting the **flags.force_snat_for_lb = 1** or **flags.skip_snat_for_lb = 1** accordingly.

- For the established traffic, a priority 50 flow that matches **ct.est && !ct.rel && !ct.new && ct_mark.natted** with an action of **next;**, if the router has load balancer assigned to it. Along with two priority 70 flows that match **skip_snat** and **force_snat** flags, setting

the **flags.force_snat_for_lb = 1** or **flags.skip_snat_for_lb = 1** accordingly.

Ingress Table 9: DNAT on Gateway Routers

- For each configuration in the OVN Northbound database, that asks to change the destination IP address of a packet from *A* to *B*, a priority−100 flow matches **ip && ip4.dst ==** *A* or **ip && ip6.dst ==** *A* with an action **flags.loopback = 1; ct_dnat(***B***);**. If the Gateway router is configured to force SNAT any DNATed packet, the above action will be replaced by **flags.force_snat_for_dnat = 1; flags.loopback = 1; ct_dnat(***B***);**. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **ip4/6.dst=** (*B*).

  If the NAT rule has **allowed_ext_ips** configured, then there is an additional match **ip4.src ==** *allowed_ext_ips* . Similarly, for IPV6, match would be **ip6.src ==** *allowed_ext_ips*.

  If the NAT rule has **exempted_ext_ips** set, then there is an additional flow configured at priority 101. The flow matches if source ip is an **exempted_ext_ip** and the action is **next;** . This flow is used to bypass the ct_dnat action for a packet originating from **exempted_ext_ips**.

  For each configuration in the OVN Northbound database, that asks to change the destination IP address of a packet from *A* to *B*, match *M* and priority *P*, a logical flow that matches **ip && ip4.dst ==** *A* or **ip && ip6.dst ==** *A* **&&** (*M*) with an action **flags.loopback = 1; ct_dnat(***B***);**. The priority of the flow is calculated based as **300 +** *P*. If the Gateway router is configured to force SNAT any DNATed packet, the above action will be replaced by **flags.force_snat_for_dnat = 1; flags.loopback = 1; ct_dnat(***B***);**. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **ip4/6.dst=** (*B*).

- If the **options:ct−commit−all** is set to **true** the following flow is configured matching on **ip && ct.new** with an action **ct_commit_to_zone(dnat);**.

- A priority−0 logical flow with match **1** has actions **next;**.

Ingress Table 9: DNAT on Distributed Routers

On distributed routers, the DNAT table only handles packets with destination IP address that needs to be DNATted from a virtual IP address to a real IP address. The unDNAT processing in the reverse direction is handled in a separate table in the egress pipeline.

- For each configuration in the OVN Northbound database, that asks to change the destination IP address of a packet from *A* to *B*, a priority−100 flow matches **ip && ip4.dst ==** *B* **&& inport ==** *GW*, where *GW* is the logical router gateway port corresponding to the NAT rule (specified or inferred), with an action **ct_dnat(***B***);**. The match will include **ip6.dst ==** *B* in the IPv6 case. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **ip4/6.dst=**(*B*).

  If the NAT rule cannot be handled in a distributed manner, then the priority−100 flow above is only programmed on the gateway chassis.

  If the NAT rule has **allowed_ext_ips** configured, then there is an additional match **ip4.src ==** *allowed_ext_ips* . Similarly, for IPV6, match would be **ip6.src ==** *allowed_ext_ips*.

  If the NAT rule has **exempted_ext_ips** set, then there is an additional flow configured at priority 101. The flow matches if source ip is an **exempted_ext_ip** and the action is **next;** . This flow is used to bypass the ct_dnat action for a packet originating from **exempted_ext_ips**.

  If the **options:ct−commit−all** is set to **true** the following flow is configured matching on **ip && ct.new && inport == DGP && is_chassis_resident(CHASSIS)** with an action **ct_commit_to_zone(dnat);**.

A priority−0 logical flow with match **1** has actions **next;**.

Ingress Table 11: Load balancing affinity learn

Load balancing affinity learn table contains the following logical flows:

- For all the configured load balancing rules for a logical router where a positive affinity timeout *T* is specified in **options** column, that includes a L4 port *PORT* of protocol *P* and IPv4 or IPv6 address *VIP*, a priority−100 flow that matches on **reg9[6] == 0 && ct.new && ip && reg0 ==** *VIP* **&&** *P* **&& reg9[16..31] ==** *PORT* (**xxreg0 ==** *VIP* in the IPv6 case) with an action of **commit_lb_aff(vip =** *VIP*:*PORT***, backend =** *backend ip***:** *backend port***, proto =** *P***, timeout =** *T***);**.
- A priority 0 flow is added which matches on all packets and applies the action **next;**.

Ingress Table 12: ECMP symmetric reply processing

- If ECMP routes with symmetric reply are configured in the **OVN_Northbound** database for a gateway router, a priority−100 flow is added for each router port on which symmetric replies are configured. The matching logic for these ports essentially reverses the configured logic of the ECMP route. So for instance, a route with a destination routing policy will instead match if the source IP address matches the static route's prefix. The flow uses the action **ct_commit { ct_label.ecmp_reply_eth = eth.src;" " ct_mark.ecmp_reply_port =** *K***;}; commit_ecmp_nh(); next;** to commit the connection and storing **eth.src** and the ECMP reply port binding tunnel key *K* in the **ct_label** and the traffic pattern to table **76** or **77**.

Ingress Table 13: IPv6 ND RA option processing

- A priority−50 logical flow is added for each logical router port configured with IPv6 ND RA options which matches IPv6 ND Router Solicitation packet and applies the action **put_nd_ra_opts** and advances the packet to the next table.

  reg0[5] = put_nd_ra_opts(*options*);next;

  For a valid IPv6 ND RS packet, this transforms the packet into an IPv6 ND RA reply and sets the RA options to the packet and stores 1 into reg0[5]. For other kinds of packets, it just stores 0 into reg0[5]. Either way, it continues to the next table.
- A priority−0 logical flow with match **1** has actions **next;**.

Ingress Table 14: IPv6 ND RA responder

This table implements IPv6 ND RA responder for the IPv6 ND RA replies generated by the previous table.

- A priority−50 logical flow is added for each logical router port configured with IPv6 ND RA options which matches IPv6 ND RA packets and **reg0[5] == 1** and responds back to the **inport** after applying these actions. If **reg0[5]** is set to 1, it means that the action **put_nd_ra_opts** was successful.

  eth.dst = eth.src;
  eth.src = *E*;
  ip6.dst = ip6.src;
  ip6.src = *I*;
  outport = *P*;
  flags.loopback = 1;
  output;

  where *E* is the MAC address and *I* is the IPv6 link local address of the logical router port.

  (This terminates packet processing in ingress pipeline; the packet does not go to the next ingress table.)

- A priority−0 logical flow with match **1** has actions **next;**.

Ingress Table 15: IP Routing Pre

If a packet arrived at this table from Logical Router Port *P* which has **options:route_table** value set, a logical flow with match **inport == "***P***"** with priority 100 and action setting unique-generated per-datapath 32-bit value (non-zero) in OVS register 7. This register's value is checked in next table. If packet didn't match any configured inport (<*main*> route table), register 7 value is set to 0.

This table contains the following logical flows:

- Priority−100 flow with match **inport == "LRP_NAME"** value and action, which set route table identifier in reg7.

  A priority−0 logical flow with match **1** has actions **reg7 = 0; next;**.

Ingress Table 16: IP Routing

A packet that arrives at this table is an IP packet that should be routed to the address in **ip4.dst** or **ip6.dst**. This table implements IP routing, setting **reg0** (or **xxreg0** for IPv6) to the next-hop IP address (leaving **ip4.dst** or **ip6.dst**, the packet's final destination, unchanged) and advances to the next table for ARP resolution. It also sets **reg1** (or **xxreg1**) to the IP address owned by the selected router port (ingress table **ARP Request** will generate an ARP request, if needed, with **reg0** as the target protocol address and **reg1** as the source protocol address).

For ECMP routes, i.e. multiple static routes with same policy and prefix but different nexthops, the above actions are deferred to next table. This table, instead, is responsible for determine the ECMP group id and select a member id within the group based on 5-tuple hashing. It stores group id in **reg8[0..15]** and member id in **reg8[16..31]**. This step is skipped with a priority−10300 rule if the traffic going out the ECMP route is reply traffic, and the ECMP route was configured to use symmetric replies. Instead, the stored values in conntrack is used to choose the destination. The **ct_label.ecmp_reply_eth** tells the destination MAC address to which the packet should be sent. The **ct_mark.ecmp_reply_port** tells the logical router port on which the packet should be sent. These values saved to the conntrack fields when the initial ingress traffic is received over the ECMP route and committed to conntrack. If **REGBIT_KNOWN_ECMP_NH** is set, the priority−10300 flows in this stage set the **outport**, while the **eth.dst** is set by flows at the ARP/ND Resolution stage.

This table contains the following logical flows:

- Priority−10550 flow that drops IPv6 Router Solicitation/Advertisement packets that were not processed in previous tables.
- Priority−10550 flows that drop IGMP and MLD packets with source MAC address owned by the router. These are used to prevent looping statically forwarded IGMP and MLD packets for which TTL is not decremented (it is always 1).
- Priority−10500 flows that match IP multicast traffic destined to groups registered on any of the attached switches and sets **outport** to the associated multicast group that will eventually flood the traffic to all interested attached logical switches. The flows also decrement TTL.
- Priority−10460 flows that match IGMP and MLD control packets, set **outport** to the **MC_STATIC** multicast group, which **ovn−northd** populates with the logical ports that have **options :mcast_flood='true'**. If no router ports are configured to flood multicast traffic the packets are dropped.
- Priority−10450 flow that matches unregistered IP multicast traffic decrements TTL and sets **outport** to the **MC_STATIC** multicast group, which **ovn−northd** populates with the logical ports that have **options :mcast_flood='true'**. If no router ports are configured to flood multicast traffic the packets are dropped.

- IPv4 routing table. For each route to IPv4 network *N* with netmask *M*, on router port *P* with IP address *A* and Ethernet address *E*, a logical flow with match **ip4.dst == *N*/*M***, whose priority is the number of 1-bits in *M*, has the following actions:

  ip.ttl−−;
  reg8[0..15] = 0;
  reg0 = *G*;
  reg1 = *A*;
  eth.src = *E*;
  outport = *P*;
  flags.loopback = 1;
  next;

  (Ingress table 1 already verified that **ip.ttl−−;** will not yield a TTL exceeded error.)

  If the route has a gateway, *G* is the gateway IP address. Instead, if the route is from a configured static route, *G* is the next hop IP address. Else it is **ip4.dst**.

- IPv6 routing table. For each route to IPv6 network *N* with netmask *M*, on router port *P* with IP address *A* and Ethernet address *E*, a logical flow with match in CIDR notation **ip6.dst ==** *N*/*M*, whose priority is the integer value of *M*, has the following actions:

  ip.ttl−−;
  reg8[0..15] = 0;
  xxreg0 = *G*;
  xxreg1 = *A*;
  eth.src = *E*;
  outport = inport;
  flags.loopback = 1;
  next;

  (Ingress table 1 already verified that **ip.ttl−−;** will not yield a TTL exceeded error.)

  If the route has a gateway, *G* is the gateway IP address. Instead, if the route is from a configured static route, *G* is the next hop IP address. Else it is **ip6.dst**.

  If the address *A* is in the link-local scope, the route will be limited to sending on the ingress port.

  For each static route the **reg7 == id &&** is prefixed in logical flow match portion. For routes with **route_table** value set a unique non-zero id is used. For routes within **<main>** route table (no route table set), this id value is 0.

  For each *connected* route (route to the LRP's subnet CIDR) the logical flow match portion has no **reg7 == id &&** prefix to have route to LRP's subnets in all routing tables.

- For ECMP routes, they are grouped by policy and prefix. An unique id (non-zero) is assigned to each group, and each member is also assigned an unique id (non-zero) within each group.

  For each IPv4/IPv6 ECMP group with group id *GID* and member ids *MID1*, *MID2*, ..., a logical flow with match in CIDR notation **ip4.dst ==** *N*/*M*, or **ip6.dst ==** *N*/*M*, whose priority is the integer value of *M*, has the following actions:

  ip.ttl−−;
  flags.loopback = 1;
  reg8[0..15] = *GID*;
  reg8[16..31] = select(*MID1*, *MID2*, ...);

However, when there is only one route in an ECMP group, group actions will be:

ip.ttl−−;
flags.loopback = 1;
reg8[0..15] = *GID*;
reg8[16..31] = *MID1*);

- A priority−0 logical flow that matches all packets not already handled (match **1**) and drops them (action **drop;**).

Ingress Table 17: IP_ROUTING_ECMP

This table implements the second part of IP routing for ECMP routes following the previous table. If a packet matched a ECMP group in the previous table, this table matches the group id and member id stored from the previous table, setting **reg0** (or **xxreg0** for IPv6) to the next-hop IP address (leaving **ip4.dst** or **ip6.dst**, the packet's final destination, unchanged) and advances to the next table for ARP resolution. It also sets **reg1** (or **xxreg1**) to the IP address owned by the selected router port (ingress table **ARP Request** will generate an ARP request, if needed, with **reg0** as the target protocol address and **reg1** as the source protocol address).

This processing is skipped for reply traffic being sent out of an ECMP route if the route was configured to use symmetric replies.

This table contains the following logical flows:

- A priority−150 flow that matches **reg8[0..15] == 0** with action **next;** directly bypasses packets of non-ECMP routes.
- For each member with ID *MID* in each ECMP group with ID *GID*, a priority−100 flow with match **reg8[0..15] ==** *GID* **&& reg8[16..31] ==** *MID* has following actions:

  [xx]reg0 = *G*;
  [xx]reg1 = *A*;
  eth.src = *E*;
  outport = *P*;

- A priority−0 logical flow that matches all packets not already handled (match **1**) and drops them (action **drop;**).

Ingress Table 18: Router policies

This table adds flows for the logical router policies configured on the logical router. Please see the **OVN_Northbound** database **Logical_Router_Policy** table documentation in **ovn−nb** for supported actions.

- For each router policy configured on the logical router, a logical flow is added with specified priority, match and actions.
- If the policy action is **reroute** with 2 or more nexthops defined, then the logical flow is added with the following actions:

  reg8[0..15] = *GID*;
  reg8[16..31] = select(1,..n);

  where *GID* is the ECMP group id generated by **ovn−northd** for this policy and *n* is the number of nexthops. **select** action selects one of the nexthop member id, stores it in the register **reg8[16..31]** and advances the packet to the next stage.
- If the policy action is **reroute** with just one nexhop, then the logical flow is added with the following actions:

[xx]reg0 = *H*;
eth.src = *E*;
outport = *P*;
reg8[0..15] = 0;
flags.loopback = 1;
next;

where *H* is the **nexthop** defined in the router policy, *E* is the ethernet address of the logical router port from which the **nexthop** is reachable and *P* is the logical router port from which the **nexthop** is reachable.

- If a router policy has the option **pkt_mark=***m* set and if the action is **not** drop, then the action also includes **pkt.mark =** *m* to mark the packet with the marker *m*.

Ingress Table 19: ECMP handling for router policies

This table handles the ECMP for the router policies configured with multiple nexthops.

- A priority−150 flow is added to advance the packet to the next stage if the ECMP group id register **reg8[0..15]** is 0.
- For each ECMP reroute router policy with multiple nexthops, a priority−100 flow is added for each nexthop *H* with the match **reg8[0..15] ==** *GID* **&& reg8[16..31] ==** *M* where *GID* is the router policy group id generated by **ovn−northd** and *M* is the member id of the nexthop *H* generated by **ovn−northd**. The following actions are added to the flow:

  [xx]reg0 = *H*;
  eth.src = *E*;
  outport = *P*
  "flags.loopback = 1; "
  "next;"

  where *H* is the **nexthop** defined in the router policy, *E* is the ethernet address of the logical router port from which the **nexthop** is reachable and *P* is the logical router port from which the **nexthop** is reachable.
- A priority−0 logical flow that matches all packets not already handled (match **1**) and drops them (action **drop;**).

Ingress Table 20: DHCP Relay Response Check

This stage process the DHCP response packets coming from the DHCP server.

- A priority 100 logical flow is added for each logical router port configured with DHCP relay that matches **ip4.src** is DHCP server ip and **ip4.dst** is lrp IP and **ip4.frag == 0** and **udp.src == 67** and **udp.dst == 67** and applies **dhcp_relay_resp_chk** action. Original destination ip is stored in reg2.

  reg9[8] = dhcp_relay_resp_chk(*lrp_ip*,
  dhcp_server_ip);next

  if action is successful then, dest mac and dest IP addresses are updated in the packet and stores 1 into reg9[8] else stores 0 into reg9[8].
- A priority−0 flow that matches all packets to advance to the next table.

Ingress Table 21: DHCP Relay Response

This stage process the DHCP response packets on which **dhcp_relay_resp_chk** action is applied in the previous stage.

- A priority 100 logical flow is added for each logical router port configured with DHCP relay that matches **ip4.src** is DHCP server ip and **reg2** is lrp IP and **udp.src == 67** and **udp.dst == 67** and **reg9[8] == 1** and applies following actions. If **reg9[8]** is set to 1 then, **dhcp_relay_resp_chk** was successful.

  ip4.src = *lrp ip*;
  udp.dst = 68;
  outport = *lrp port*;
  output;

- A priority 1 logical flow is added for the logical router port on which DHCP relay is enabled that matches **ip4.src** is DHCP server ip and **reg2** is lrp IP and **udp.src == 67** and **udp.dst == 67** and **reg9[8] == 0** and drops the packet. If **reg9[8]** is set to 0 then, **dhcp_relay_resp_chk** was unsuccessful.
- A priority−0 flow that matches all packets to advance to the next table.

Ingress Table 22: ARP/ND Resolution

Any packet that reaches this table is an IP packet whose next-hop IPv4 address is in **reg0** or IPv6 address is in **xxreg0**. (**ip4.dst** or **ip6.dst** contains the final destination.) This table resolves the IP address in **reg0** (or **xxreg0**) into an output port in **outport** and an Ethernet address in **eth.dst**, using the following flows:

- A priority−500 flow that matches IP multicast traffic that was allowed in the routing pipeline. For this kind of traffic the **outport** was already set so the flow just advances to the next table.
- Priority−200 flows that match ECMP reply traffic for the routes configured to use symmetric replies, with actions **push(xxreg1); xxreg1 = ct_label; eth.dst = xxreg1[32..79]; pop(xxreg1); next;**. **xxreg1** is used here to avoid masked access to ct_label, to make the flow HW-offloading friendly.
- Static MAC bindings. MAC bindings can be known statically based on data in the **OVN_Northbound** database. For router ports connected to logical switches, MAC bindings can be known statically from the **addresses** column in the **Logical_Switch_Port** table. (Note: the flow is not installed for IPs of logical switch ports of type **virtual**, and dynamic MAC binding is used for those IPs instead, so that virtual parent failover does not depend on **ovn−northd**, to achieve better failover performance.) For router ports connected to other logical routers, MAC bindings can be known statically from the **mac** and **networks** column in the **Logical_Router_Port** table. (Note: the flow is NOT installed for the IP addresses that belong to a neighbor logical router port if the current router has the **options:dynamic_neigh_routers** set to **true**)

  For each IPv4 address *A* whose host is known to have Ethernet address *E* on router port *P*, a priority−100 flow with match **outport ===** *P* **&& reg0 ==** *A* has actions **eth.dst =** *E***; next;**.

  For each IPv6 address *A* whose host is known to have Ethernet address *E* on router port *P*, a priority−100 flow with match **outport ===** *P* **&& xxreg0 ==** *A* has actions **eth.dst =** *E***; next;**.

  For each logical router port with an IPv4 address *A* and a mac address of *E* that is reachable via a different logical router port *P*, a priority−100 flow with match **outport ===** *P* **&& reg0 ==** *A* has actions **eth.dst =** *E***; next;**.

  For each logical router port with an IPv6 address *A* and a mac address of *E* that is reachable via a different logical router port *P*, a priority−100 flow with match **outport ===** *P* **&& xxreg0 ==** *A* has actions **eth.dst =** *E***; next;**.

- Static MAC bindings from NAT entries. MAC bindings can also be known for the entries in the **NAT** table. Below flows are programmed for distributed logical routers i.e with a distributed router port.

  For each row in the **NAT** table with IPv4 address *A* in the **external_ip** column of **NAT** table, below two flows are programmed:

  A priority−100 flow with the match **outport ==** *P* **&& reg0 ==** *A* has actions **eth.dst =** *E***; next;**, where **P** is the distributed logical router port, *E* is the Ethernet address if set in the **external_mac** column of **NAT** table for of type **dnat_and_snat**, otherwise the Ethernet address of the distributed logical router port. Note that if the **external_ip** is not within a subnet on the owning logical router, then OVN will only create ARP resolution flows if the **options:add_route** is set to **true**. Otherwise, no ARP resolution flows will be added.

  Corresponding to the above flow, a priority−150 flow with the match **inport ==** *P* **&& outport ==** *P* **&& ip4.dst ==** *A* has actions **drop;** to exclude packets that have gone through DNAT/unSNAT stage but failed to convert the destination, to avoid loop.

  For IPv6 NAT entries, same flows are added, but using the register **xxreg0** and field **ip6** for the match.

- If the router datapath runs a port with **redirect−type** set to **bridged**, for each distributed NAT rule with IP *A* in the **logical_ip** column and logical port *P* in the **logical_port** column of **NAT** table, a priority−90 flow with the match **outport ==** *Q* **&& ip.src ===** *A* **&& is_chassis_resident**(*P*), where **Q** is the distributed logical router port and action **get_arp(outport, reg0); next;** for IPv4 and **get_nd(outport, xxreg0); next;** for IPv6.

- Traffic with IP destination an address owned by the router should be dropped. Such traffic is normally dropped in ingress table **IP Input** except for IPs that are also shared with SNAT rules. However, if there was no unSNAT operation that happened successfully until this point in the pipeline and the destination IP of the packet is still a router owned IP, the packets can be safely dropped.

  A priority−2 logical flow with match **ip4.dst = {..}** matches on traffic destined to router owned IPv4 addresses which are also SNAT IPs. This flow has action **drop;**.

  A priority−2 logical flow with match **ip6.dst = {..}** matches on traffic destined to router owned IPv6 addresses which are also SNAT IPs. This flow has action **drop;**.

  A priority−0 logical that flow matches all packets not already handled (match **1**) and drops them (action **drop;**).

- Dynamic MAC bindings. These flows resolve MAC-to-IP bindings that have become known dynamically through ARP or neighbor discovery. (The ingress table **ARP Request** will issue an ARP or neighbor solicitation request for cases where the binding is not yet known.)

  A priority−0 logical flow with match **ip4** has actions **get_arp(outport, reg0); next;**.

  A priority−0 logical flow with match **ip6** has actions **get_nd(outport, xxreg0); next;**.

- For a distributed gateway LRP with **redirect−type** set to **bridged**, a priority−50 flow will match **outport == "ROUTER_PORT" and !is_chassis_resident ("cr−ROUTER_PORT")** has actions **eth.dst =** *E***; next;**, where *E* is the ethernet address of the logical router port.

Ingress Table 23: Check packet length

For distributed logical routers or gateway routers with gateway port configured with **options:gateway_mtu** to a valid integer value, this table adds a priority−50 logical flow with the match **outport ==** *GW_PORT* where *GW_PORT* is the gateway router port and applies the actions **check_pkt_larger** and **ct_state_save** and then advances the packet to the next table.

REGBIT_PKT_LARGER = check_pkt_larger(*L*);

REG_CT_STATE = ct_state_save();
next;

where *L* is the packet length to check for. If the packet is larger than *L*, it stores 1 in the register bit **REGBIT_PKT_LARGER**. The value of *L* is taken from **options:gateway_mtu** column of **Logical_Router_Port** row.

If the port is also configured with **options:gateway_mtu_bypass** then another flow is added, with priority−55, to bypass the **check_pkt_larger** flow.

This table adds one priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 24: Handle larger packets

For distributed logical routers or gateway routers with gateway port configured with **options:gateway_mtu** to a valid integer value, this table adds the following priority−150 logical flow for each logical router port with the match **inport ==** *LRP* **&& outport ==** *GW_PORT* **&& REGBIT_PKT_LARGER && !REGBIT_EGRESS_LOOPBACK**, where *LRP* is the logical router port and *GW_PORT* is the gateway port and applies the following action for ipv4 and ipv6 respectively:

icmp4 {
icmp4.type = 3; /* Destination Unreachable. */
icmp4.code = 4; /* Frag Needed and DF was Set. */
icmp4.frag_mtu = *M*;
eth.dst = *E*;
ip4.dst = ip4.src;
ip4.src = *I*;
ip.ttl = 255;
REGBIT_EGRESS_LOOPBACK = 1;
REGBIT_PKT_LARGER = 0;
next(pipeline=ingress, table=0);
};
icmp6 {
icmp6.type = 2;
icmp6.code = 0;
icmp6.frag_mtu = *M*;
eth.dst = *E*;
ip6.dst = ip6.src;
ip6.src = *I*;
ip.ttl = 255;
REGBIT_EGRESS_LOOPBACK = 1;
REGBIT_PKT_LARGER = 0;
next(pipeline=ingress, table=0);
};

- Where *M* is the (fragment MTU - 58) whose value is taken from **options:gateway_mtu** column of **Logical_Router_Port** row.
- *E* is the Ethernet address of the logical router port.
- *I* is the IPv4/IPv6 address of the logical router port.

This table adds one priority−0 fallback flow that matches all packets and advances to the next table.

Ingress Table 25: Gateway Redirect

For distributed logical routers where one or more of the logical router ports specifies a gateway chassis, this table redirects certain packets to the distributed gateway port instances on the gateway chassises. This table has the following flows:

- For all the configured load balancing rules that include an IPv4 address *VIP*, and a list of IPv4 backend addresses *B0*, *B1* .. *Bn* defined for the *VIP* a priority−200 flow is added that matches **ip4 && (ip4.src ==** *B0* **|| ip4.src ==** *B1* **|| ... || ip4.src ==** *Bn***)** with an action **outport =** *CR***; next;** where *CR* is the **chassisredirect** port representing the instance of the logical router distributed gateway port on the gateway chassis. If the backend IPv4 address *Bx* is also configured with L4 port *PORT* of protocol *P*, then the match also includes **P.src** == *PORT*. Similar flows are added for IPv6.
- For each NAT rule in the OVN Northbound database that can be handled in a distributed manner, a priority−100 logical flow with match **ip4.src ==** *B* **&& outport ==** *GW* && is_chassis_resident(*P*), where *GW* is the distributed gateway port specified in the NAT rule and *P* is the NAT logical port. IP traffic matching the above rule will be managed locally setting **reg1** to *C* and **eth.src** to *D*, where *C* is NAT external ip and *D* is NAT external mac.
- For each **dnat_and_snat** NAT rule with **stateless=true** and **allowed_ext_ips** configured, a priority−75 flow is programmed with match **ip4.dst ==** *B* and action **outport =** *CR***; next;** where *B* is the NAT rule external IP and *CR* is the **chassisredirect** port representing the instance of the logical router distributed gateway port on the gateway chassis. Moreover a priority−70 flow is programmed with same match and action **drop;**. For each **dnat_and_snat** NAT rule with **stateless=true** and **exempted_ext_ips** configured, a priority−75 flow is programmed with match **ip4.dst ==** *B* and action **drop;** where *B* is the NAT rule external IP. A similar flow is added for IPv6 traffic.
- For each NAT rule in the OVN Northbound database that can be handled in a distributed manner, a priority−80 logical flow with drop action if the NAT logical port is a virtual port not claimed by any chassis yet.
- A priority−50 logical flow with match **outport ==** *GW* has actions **outport =** *CR***; next;**, where *GW* is the logical router distributed gateway port and *CR* is the **chassisredirect** port representing the instance of the logical router distributed gateway port on the gateway chassis.
- A priority−0 logical flow with match **1** has actions **next;**.

Ingress Table 26: Network ID

This table contains flows that set **flags.network_id** for IP packets:

- A priority−110 flow with match:
  - for IPv4: **outport ==** *P* **&& REG_NEXT_HOP_IPV4 ==** *I*/*C* **&& ip4**
  - for IPv6: **outport ==** *P* **&& REG_NEXT_HOP_IPV6 ==** *I*/*C* **&& ip6**

  and actions **flags.network_id =** *N***; next;**. Where *P* is the outport, *I*/*C* is a network CIDR of the port *P*, and *N* is the network id (index). There is one flow like this per router port's network.

  flags.network_id is 4 bits, and thus only 16 networks can be indexed. If the number of networks is greater than 16, networks 17 and up will have the actions **flags.network_id = 0; next;** and only the lexicographically first IP will be considered for SNAT for those networks.
- A lower priority−105 flow with match **1** and actions **flags.network_id = 0; next;**. This is for the case that the next-hop doesn't belong to any of the port networks, so **flags.network_id** should be set to zero.
- Catch-all: A priority−0 flow with match **1** has actions **next;**.

Ingress Table 27: ARP Request

In the common case where the Ethernet destination has been resolved, this table advances the packet to the next table. Otherwise, it composes and sends an ARP or IPv6 Neighbor Solicitation request. It holds the

following flows:

- Unknown MAC address. A priority−100 flow for IPv4 packets with match **eth.dst == 00:00:00:00:00:00** has the following actions:

  ```
  arp {
    eth.dst = ff:ff:ff:ff:ff:ff;
    arp.spa = reg1;
    arp.tpa = reg0;
    arp.op = 1;  /* ARP request. */
    output;
  };
  ```

  Unknown MAC address. A priority−100 flow for IPv6 packets with match **eth.dst == 00:00:00:00:00:00** has the following actions:

  ```
  nd_ns {
    nd.target = xxreg0;
    output;
  };
  ```

  (Ingress table **IP Routing** initialized **reg1** with the IP address owned by **outport** and **(xx)reg0** with the next-hop IP address)

  The IP packet that triggers the ARP/IPv6 NS request is dropped.

- Known MAC address. A priority−0 flow with match **1** has actions **next;**.

Ingress Table 28: ECMP symmetric reply processing for egress

This table contains logical flows that commit IP traffic forwarded by ECMP symmetric reply static routes in the "route direction", that is, for sessions initiated from behind such routes. These flows can be hit only on gateway routers, the only type of routers that supports ECMP symmetric reply routes. As the egress port of the traffic needs to be stored in conntrack for these sessions, one logical flow is added for each logical router port.

Egress Table 0: Check DNAT local

This table checks if the packet needs to be DNATed in the router ingress table **lr_in_dnat** after it is SNATed and looped back to the ingress pipeline. This check is done only for routers configured with distributed gateway ports and NAT entries. This check is done so that SNAT and DNAT is done in different zones instead of a common zone.

- A priority−0 logical flow with match **1** has actions **REGBIT_DST_NAT_IP_LOCAL = 0; next;**.

Egress Table 1: UNDNAT

This is for already established connections' reverse traffic. i.e., DNAT has already been done in ingress pipeline and now the packet has entered the egress pipeline as part of a reply. This traffic is unDNATed here.

- A priority−0 logical flow with match **1** has actions **next;**.

Egress Table 1: UNDNAT on Gateway Routers

- For IPv6 Neighbor Discovery or Router Solicitation/Advertisement traffic, a priority−100 flow with action **next;**.
- For all IP packets, a priority−50 flow with an action **flags.loopback = 1; ct_dnat;**.

Egress Table 1: UNDNAT on Distributed Routers

- For all the configured load balancing rules for a router with gateway port in **OVN_Northbound** database that includes an IPv4 address **VIP**, for every backend IPv4 address *B* defined for the **VIP** a priority−120 flow is programmed on gateway chassis that matches **ip && ip4.src ==** *B* **&& outport ==** *GW*, where *GW* is the logical router gateway port with an action **ct_dnat;**. If the backend IPv4 address *B* is also configured with L4 port *PORT* of protocol *P*, then the match also includes **P.src** == *PORT*. These flows are not added for load balancers with IPv6 *VIPs*.

  If the router is configured to force SNAT any load-balanced packets, above action will be replaced by **flags.force_snat_for_lb = 1; ct_dnat;**.

- For each configuration in the OVN Northbound database that asks to change the destination IP address of a packet from an IP address of *A* to *B*, a priority−100 flow matches **ip && ip4.src ==** *B* **&& outport ==** *GW*, where *GW* is the logical router gateway port, with an action **ct_dnat;**. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **next;**.

  If the NAT rule cannot be handled in a distributed manner, then the priority−100 flow above is only programmed on the gateway chassis with the action **ct_dnat**.

  If the NAT rule can be handled in a distributed manner, then there is an additional action **eth.src =** *EA***;**, where *EA* is the ethernet address associated with the IP address *A* in the NAT rule. This allows upstream MAC learning to point to the correct chassis.

Egress Table 2: Post UNDNAT

- A priority−70 logical flow is added that initiates CT state for traffic that is configured to be SNATed on Distributed routers. This allows the next table, **lr_out_snat**, to effectively match on various CT states.
- A priority−50 logical flow is added that commits any untracked flows from the previous table **lr_out_undnat** for Gateway routers. This flow matches on **ct.new && ip** with action **ct_commit { } ; next;** .
- If the **options:ct−commit−all** is set to **true** the following flows are configured matching on **ip && (!ct.trk || !ct.rpl) && flags.unsnat_not_tracked == 1** with an action **ct_next(snat);** and **ip && flags.unsnat_new == 1** with an action **next;**. There is extra match when there is configured DGP **outport == DGP && is_chassis_resident(CHASSIS)**.
- A priority−0 logical flow with match **1** has actions **next;**.

Egress Table 3: SNAT

Packets that are configured to be SNATed get their source IP address changed based on the configuration in the OVN Northbound database.

- A priority−120 flow to advance the IPv6 Neighbor solicitation packet to next table to skip SNAT. In the case where ovn-controller injects an IPv6 Neighbor Solicitation packet (for **nd_ns** action) we don't want the packet to go through conntrack.

Egress Table 3: SNAT on Gateway Routers

- If the Gateway router in the OVN Northbound database has been configured to force SNAT a packet (that has been previously DNATted) to *B*, a priority−100 flow matches **flags.force_snat_for_dnat == 1 && ip** with an action **ct_snat(***B***);**.
- If a load balancer configured to skip snat has been applied to the Gateway router pipeline, a priority−120 flow matches **flags.skip_snat_for_lb == 1 && ip** with an action **next;**.
- If the Gateway router in the OVN Northbound database has been configured to force SNAT a packet (that has been previously load-balanced) using router IP (i.e **options**:lb_force_snat_ip=router_ip), then for each logical router port *P* attached to the Gateway router, and for each network configured for this port, a priority−110 flow

matches **flags.force_snat_for_lb == 1** && **ip4** && **flags.network_id ==** *N* && **outport ==** *P*, where *N* is the network index, with an action **ct_snat(***R***);** where *R* is the IP configured on the router port. A similar flow is created for IPv6, with **ip6** instead of **ip4**. *N*, the network index, will be 0 for networks 17 and up.

If the logical router port *P* is configured with multiple IPv4 and multiple IPv6 addresses, the IPv4 and IPv6 address within the same network as the next-hop will be chosen as *R* for SNAT. However, if there are more than 16 networks configured, the lexicographically first IP will be considered for SNAT for networks 17 and up.

- A priority−105 flow matches the old behavior for if northd is upgraded before controller and **flags.network_id** is not recognized. It is only added if there's at least one network configured (excluding LLA for IPv6). It matches on: **flags.force_snat_for_lb == 1** && **ip4** && **outport ==** *P*, with action: **ct_snat(***R***)**. *R* is the lexicographically first IP address configured. There is a similar flow for IPv6 with **ip6** instead of **ip4**.
- If the Gateway router in the OVN Northbound database has been configured to force SNAT a packet (that has been previously load-balanced) to *B*, a priority−100 flow matches **flags.force_snat_for_lb == 1 && ip** with an action **ct_snat(***B***);**.
- For each configuration in the OVN Northbound database, that asks to change the source IP address of a packet from an IP address of *A* or to change the source IP address of a packet that belongs to network *A* to *B*, a flow matches **ip && ip4.src ==** *A* **&& (!ct.trk || !ct.rpl)** with an action **ct_snat(***B***);**. The priority of the flow is calculated based on the mask of *A*, with matches having larger masks getting higher priorities. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **ip4/6.src= (***B***)**.

  For each configuration in the OVN Northbound database, that asks to change the source IP address of a packet from an IP address of *A* or to change the source IP address of a packet that belongs to network *A* to *B*, match *M* and priority *P*, a flow matches **ip && ip4.src ==** *A* **&& (!ct.trk || !ct.rpl) && (***M***)** with an action **ct_snat(***B***);** . The priority of the flow is calculated based as **300 +** *P*. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **ip4/6.src=(***B***)**.
- If the NAT rule has **allowed_ext_ips** configured, then there is an additional match **ip4.dst ==** *allowed_ext_ips* . Similarly, for IPV6, match would be **ip6.dst ==** *allowed_ext_ips*.
- If the NAT rule has **exempted_ext_ips** set, then there is an additional flow configured at the priority + 1 of corresponding NAT rule. The flow matches if destination ip is an **exempted_ext_ip** and the action is **next;** . This flow is used to bypass the ct_snat action for a packet which is destinted to **exempted_ext_ips**.
- If the **options:ct−commit−all** is set to **true** the following two flows are configured matching on **ip && (!ct.trk || !ct.rpl) && flags.unsnat_new == 1** and **ip && ct.new && flags.unsnat_not_tracked == 1**
  both with an action **ct_commit_to_zone(snat);**.
- A priority−0 logical flow with match **1** has actions **next;**.

Egress Table 3: SNAT on Distributed Routers

- For each configuration in the OVN Northbound database, that asks to change the source IP address of a packet from an IP address of *A* or to change the source IP address of a packet that belongs to network *A* to *B*, two flows are added. The priority *P* of these flows are calculated based on the mask of *A*, with matches having larger masks getting higher priorities.

  If the NAT rule cannot be handled in a distributed manner, then the below flows are only programmed on the gateway chassis increasing flow priority by 128 in order to be run first.

- The first flow is added with the calculated priority *P* and match **ip && ip4.src == *A* && outport == *GW***, where *GW* is the logical router gateway port, with an action **ct_snat(*B*);** to SNATed in the common zone. If the NAT rule is of type dnat_and_snat and has **stateless=true** in the options, then the action would be **ip4/6.src=(*B*)**.

  If the NAT rule can be handled in a distributed manner, then there is an additional action (for both the flows) **eth.src = *EA*;**, where *EA* is the ethernet address associated with the IP address *A* in the NAT rule. This allows upstream MAC learning to point to the correct chassis.

  If the NAT rule has **allowed_ext_ips** configured, then there is an additional match **ip4.dst == *allowed_ext_ips*** . Similarly, for IPV6, match would be **ip6.dst == *allowed_ext_ips***.

  If the NAT rule has **exempted_ext_ips** set, then there is an additional flow configured at the priority ***P* + 2** of corresponding NAT rule. The flow matches if destination ip is an **exempted_ext_ip** and the action is **next;** . This flow is used to bypass the ct_snat action for a flow which is destinted to **exempted_ext_ips**.

- An additional flow is added for traffic that goes in opposite direction (i.e. it enters a network with configured SNAT). Where the flow above matched on **ip4.src == *A* && outport == *GW***, this flow matches on **ip4.dst == *A* && inport == *GW***. A CT state is initiated for this traffic so that the following table, **lr_out_post_snat**, can identify whether the traffic flow was initiated from the internal or external network.

- If the **options:ct−commit−all** is set to **true** the following two flows are configured matching on **ip && (!ct.trk || !ct.rpl) && flags.unsnat_new == 1 && outport == DGP && is_chassis_resident(CHASSIS)**
  and **ip && ct.new && flags.unsnat_not_tracked == 1 && outport == DGP && is_chassis_resident(CHASSIS)**both with an action **ct_commit_to_zone(snat);**.

- A priority−0 logical flow with match **1** has actions **next;**.

Egress Table 4: Post SNAT

Packets reaching this table are processed according to the flows below:

- Traffic that goes directly into a network configured with SNAT on Distributed routers, and was initiated from an external network (i.e. it matches **ct.new**), is committed to the SNAT CT zone. This ensures that replies returning from the SNATed network do not have their source address translated. For details about match rules and priority see section "Egress Table 3: SNAT on Distributed Routers".

- A priority−0 logical flow that matches all packets not already handled (match **1**) and action **next;**.

Egress Table 5: Egress Loopback

For distributed logical routers where one of the logical router ports specifies a gateway chassis.

While UNDNAT and SNAT processing have already occurred by this point, this traffic needs to be forced through egress loopback on this distributed gateway port instance, in order for UNSNAT and DNAT processing to be applied, and also for IP routing and ARP resolution after all of the NAT processing, so that the packet can be forwarded to the destination.

This table has the following flows:

- For each NAT rule in the OVN Northbound database on a distributed router, a priority−100 logical flow with match **ip4.dst == *E* && outport == *GW* && is_chassis_resident(*P*)**, where *E* is the external IP address specified in the NAT rule, *GW* is the distributed gateway port corresponding to the NAT rule (specified or inferred). For dnat_and_snat NAT rule, *P* is the logical port specified in the NAT rule. If **logical_port** column of **NAT** table is NOT set, then *P* is the **chassisredirect port** of *GW* with the

following actions:

```
clone {
   ct_clear;
   inport = outport;
   outport = "";
   flags = 0;
   flags.loopback = 1;
   reg0 = 0;
   reg1 = 0;
   ...
   reg9 = 0;
   REGBIT_EGRESS_LOOPBACK = 1;
   next(pipeline=ingress, table=0);
};
```

flags.loopback is set since in_port is unchanged and the packet may return back to that port after NAT processing. **REGBIT_EGRESS_LOOPBACK** is set to indicate that egress loopback has occurred, in order to skip the source IP address check against the router address.

- A priority−0 logical flow with match **1** has actions **next;**.

Egress Table 6: Delivery

Packets that reach this table are ready for delivery. It contains:

- Priority−110 logical flows that match IP multicast packets on each enabled logical router port and modify the Ethernet source address of the packets to the Ethernet address of the port and then execute action **output;**.
- Priority−100 logical flows that match packets on each enabled logical router port, with action **output;**.
- A priority−0 logical flow that matches all packets not already handled (match **1**) and drops them (action **drop;**).

DROP SAMPLING

As described in the previous section, there are several places where ovn-northd might decided to drop a packet by explicitly creating a **Logical_Flow** with the **drop;** action.

When debug drop-sampling has been cofigured in the OVN Northbound database, the ovn-northd will replace all the **drop;** actions with a **sample(priority=65535, collector_set=***id***, obs_domain=***obs_id***, obs_point=@cookie**) action, where:

- *id* is the value the **debug_drop_collector_set** option configured in the OVN Northbound.
- *obs_id* has it's 8 most significant bits equal to the value of the **debug_drop_domain_id** option in the OVN Northbound and it's 24 least significant bits equal to the datapath's tunnel key.